PROYECTO FINAL AUTÓMATAS Y LENGUAJES FORMALES

ER-AFD - CONVERTIDOR DE ER A AFD / AFND

Integrantes:

Daniel Fernando Morales Alvizures / 5990-24-1245

Yohana Sofia Samayoa Arias / 5990-24-6796

Jhonatan Francisco Moreira Turcios / 1990-24-5115

Ernesto Emanuel Santizo Santizo / 1990-24-1077

Universidad Mariano Gálvez de Guatemala

Ingeniería en Sistemas de la Información y Ciencias de la Computación

Autómatas y Lenguajes Formales

Ingeniero Edgar René Ornelis Hoil

Noviembre 7, 2026

Índice

1. Introducción 4
2. Objetivos 5
2.1. Objetivo General 5
2.2. Objetivos Específicos 5
3. Reporte Técnico del Proyecto ER-AFD 6
3.1. Entrada, validación y árbol de sintaxis abstracta (AST) 6
3.2. Construcción del AFND mediante Thompson 8
3.3. Conversión del AFND a AFD por subconjuntos 9
3.4. Simulación de una cadena de entrada 10
3.5. Relación de módulos y archivos consultados 11
3.6. Evidencias de ejecución 11
3.7. Consideraciones técnicas y límites de la ejecución 14
4. Informe de Verificación: Casos de Prueba y Resultados 15
4.1. Propósito 15
4.2. Resumen de resultados 15
4.3. Evidencias de ejecución 15
4.4. Tabla de casos de pruebas 23
4.5. Observaciones 29
4.6. Consideraciones técnicas y límites de la ejecución 29
4.7. Conclusión técnica 29
5. Manual de usuario: Guía rápida de instalación y ejecución 30
5.1. Requisitos 30
5.2. Descargar el repositorio 31
5.3. Ingreso a la carpeta del proyecto 31
5.4. Configuración del entorno – make config 32
5.5. Construcción del proyecto – make build 33
5.6. Ejecución del programa – make run 34
5.7. Ingreso de una expresión regular y otra cadena 35
5.8. Ejecución de las pruebas – make test 37
5.9. Limpieza del proyecto – make clean 38
5.10. Interpretación de los resultados 40

5.11. Uso de comillas y espacios en las entradas 41
5.12. Finalización 42
10. Conclusiones 43
11. Referencias 44

1. Introducción

En el ámbito de la teoría de la computación y el diseño de compiladores, el procesamiento de lenguajes regulares mediante expresiones regulares (ER) y autómatas finitos representa una técnica fundamental para el análisis léxico y el reconocimiento de patrones. Si bien las expresiones regulares ofrecen una representación concisa y expresiva para definir patrones formales, su evaluación eficiente en entornos computacionales requiere su transformación a estructuras ejecutables como los Autómatas Finitos Deterministas (AFD).

El presente proyecto consiste en el desarrollo de un sistema ejecutable mediante interfaz de línea de comandos (CLI) en Python sobre entornos Linux 64-bit. La herramienta permite procesar expresiones regulares compuestas por operadores básicos (concatenación, unión |, cerradura de Kleene *), agrupamientos y secuencias de escape especiales (tales como rangos [a-z], clases de caracteres y códigos ASCII en hexadecimal). A través de la implementación explícita y manual de algoritmos teóricos (construcción de Thompson para el AFND y construcción de subconjuntos para la determinización a AFD, sin el uso de librerías nativas de regex), el programa genera y muestra las tablas de transiciones correspondientes y simula el recorrido paso a paso de cadenas de entrada para validar su aceptación o rechazo dentro del lenguaje definido.

Además, el desarrollo del sistema fue concebido bajo un enfoque riguroso de desarrollo de software utilizando Python en entornos Linux de 64 bits. Para garantizar la mantenibilidad y verificación del código, la herramienta prescinde por completo de librerías nativas de procesamiento sintáctico de expresiones regulares, recurriendo en su lugar a una estructura modular alimentada por un archivo Makefile que automatiza las tareas de configuración, construcción y ejecución. Este marco metodológico no solo asegura el cumplimiento de las restricciones de implementación, sino que también proporciona una suite de pruebas exhaustiva para medir con precisión el rendimiento y la tasa de acierto del autómata frente a diversas cadenas de entrada ASCII.

2. Objetivos

2.1. Objetivo General

Explicar cómo el sistema recibe y analiza una expresión regular, produce una representación estructurada de sus operaciones, construye un autómata finito no determinista (AFND), lo transforma en un autómata finito determinista (AFD) y utiliza este último para decidir si una cadena pertenece al lenguaje definido por la expresión. La explicación sigue el código proporcionado por el equipo, sin usar una librería de expresiones regulares para reconocer las cadenas.

2.2. Objetivos Específicos

- Implementar el módulo de lectura y parsing de expresiones regulares: Desarrollar un analizador capaz de validar e interpretar sintácticamente expresiones regulares que incluyan operadores de concatenación implícita, unión (|), cerradura de Kleene (*), agrupamiento (()), clases de conjuntos ([...], [^...]) y secuencias de escape especiales (\d, \l, \a, \z, \hHEX).

- Construir el motor de conversión ER $\rightarrow$ AFND $\rightarrow$ AFD: Aplicar de forma explícita el algoritmo de Thompson para transformar la expresión regular en un Autómata Finito No Determinista (AFND) y el algoritmo de construcción de subconjuntos para obtener su equivalente Autómata Finito Determinista (AFD) sin recurrir a librerías externas de procesamiento de expresiones regulares.

- Generar y visualizar la estructura matricial de transiciones: Desplegar en la consola de comandos las tablas de transiciones, estados iniciales, alfabeto y conjuntos de estados de aceptación tanto para el AFND (en formato q_i) como para el AFD (en formato Q_i).

- Evaluar y simular la aceptación de cadenas de entrada: Desarrollar el módulo de análisis que procese cadenas de texto ASCII a través del AFD generado, mostrando la traza paso a paso del recorrido de estados y retornando el resultado final de evaluación ("Cadena aceptada" con código de salida 1 o "Cadena rechazada" con código de salida 0).

3. Reporte Técnico del Proyecto ER-AFD

El programa funciona en línea de comandos y recibe dos argumentos: la expresión regular (ER) y la cadena que se desea comprobar. La ruta principal está en er_afd/cli.py. A partir de esos datos se encadenan cuatro módulos: el analizador, la construcción del AFND, la conversión al AFD y el simulador.

Figura 1. Flujo general del convertidor ER-AFD



Fuente: elaboración propia a partir de los módulos analizador.py, afnd.py, afd.py, simulador.py y cli.py.

El orden de ejecución es importante: no se puede construir el AFND antes de interpretar la expresión, y no se debe evaluar una cadena con el AFD antes de generar sus estados y transiciones.

3.1. Entrada, validación y árbol de sintaxis abstracta (AST)

El punto de partida del modelo Entidad-Relación es definir exactamente qué elementos de la base de datos vamos a monitorear en un momento dado.

- **A) Datos recibidos:**

El formato requerido es er-afd <ER> <cadena>. Por ejemplo, se puede utilizar la expresión (a|b)*abb con la cadena abb. El primer argumento describe el patrón; el segundo representa el texto que será aceptado o rechazado. Cuando hay espacios, se debe conservar el argumento entre comillas en la terminal.

```
make run ER='(a|b)*abb' CADENA='abb'
```

El programa acepta concatenación implícita, unión (|), cerradura de Kleene (*), paréntesis, el comodín (.), épsilon (ε), conjuntos [abc], negaciones [^abc], rangos [a-z] y escapes. Entre los escapes especiales se encuentran \N para dígitos, \A para mayúsculas, \a para minúsculas, \z para letras y \h seguido de dos dígitos hexadecimales.

- **B) Revisión sintáctica y precedencia:**

El archivo er_afd/analizador.py implementa un analizador descendente recursivo. Sus funciones leen la expresión carácter por carácter y la van descomponiendo según su prioridad: primero los paréntesis, después la repetición con *, luego la concatenación y finalmente la unión. Esta organización permite distinguir, por ejemplo, que a|bc* representa a o bien b seguido de cero o más c, y no la repetición de toda la unión.

Durante el proceso, el analizador detecta errores como expresiones vacías, paréntesis incompletos, conjuntos vacíos, rangos invertidos o códigos hexadecimales mal escritos. Si detecta una expresión inválida, muestra un mensaje y una posición aproximada en la que se encontró el problema. No utiliza la librería re de Python.

- **C) ¿Qué es el AST y para qué sirve?**

AST significa árbol de sintaxis abstracta. Es una estructura que representa las operaciones y su relación, en lugar de mantener la expresión solamente como texto. En el proyecto se construye con diccionarios de Python que incluyen una clave «tipo»; según el nodo, pueden incorporar `«valor»`, `«hijo»`, `«hijos»`, `«caracteres»` o `«negado»`.

Con la expresión (a|b)*abb, la operación principal es una concatenación. Dentro de ella existe un nodo de Kleene cuyo hijo es una unión entre a y b, seguido por los símbolos a, b y b. Gracias a esta estructura, el módulo de Thompson sabe qué operación debe construir en cada paso sin volver a analizar los caracteres originales.

- **D) Uso de listas y diccionarios:**

Los diccionarios permiten identificar claramente las propiedades de cada nodo, estado o transición mediante nombres como «origen», «símbolo» y «destino». Las listas se utilizan para almacenar los hijos de una operación, las transiciones, los estados, los alfabetos y los recorridos. Estas estructuras estándar son fáciles de recorrer, agregar y consultar y evitan depender de una biblioteca especial de expresiones regulares.

3.2. Construcción del AFND mediante Thompson

Una vez obtenido el AST, el archivo er_afd/afnd.py aplica el algoritmo de Thompson. Este método construye fragmentos pequeños de autómata y la combina de acuerdo con los nodos del árbol. Cada fragmento tiene un estado de inicio, un estado final y sus transiciones. Al finalizar, se obtiene un AFND completo.

1. Símbolo individual: se crean dos estados conectados por una transición etiquetada con ese símbolo. Los conjuntos pueden producir varias transiciones entre el mismo par de estados.
2. Unión: se agrega un inicio y un final nuevos, conectando por épsilon las alternativas. El autómata puede seguir cualquiera de las ramas.
3. Concatenación: se conecta el estado final del primer fragmento con el inicio del siguiente por una transición épsilon.
4. Cerradura de Kleene: se incorporan caminos épsilon que permiten omitir el fragmento o repetirlo tantas veces como sea necesario.

- **A) Estados y transiciones épsilon:**

El sistema asigna los nombres q0, q1, q2, etc., utilizando un contador numérico que comienza en cero. En el código los estados se almacenan como enteros y, al imprimirlos, se agrega la letra q. El estado inicial del AFND es q0; el estado de aceptación es el extremo final del fragmento completo.

Una transición épsilon (ε) permite cambiar de estado sin consumir ningún carácter de la cadena. Esto resulta esencial para enlazar las ramas de unión y las repeticiones. En el AFND puede haber varias opciones desde un mismo estado o para un mismo símbolo, por eso se le llama no determinista.

- **B) Ejemplo real del AFND generado:**

Para la ER (a|b)*abb, la ejecución del código del repositorio produce 14 estados, de q0 a q13, un alfabeto {a, b, ε} y el estado final q13. La tabla siguiente reproduce las transiciones agrupadas que presenta el programa.

Estado actual	Símbolo	Estado(s) destino
q0	ε	{q2,q1}
q2	ε	{q4,q6}
q4	a	{q5}
q5	ε	{q3}

Estado actual	Símbolo	Estado(s) destino
q6	b	{q7}
q7	ε	{q3}
q3	ε	{q2,q1}
q1	ε	{q8}
q8	a	{q9}
q9	ε	{q10}
q10	b	{q11}
q11	ε	{q12}
q12	b	{q13}

Se observa cómo la parte (a|b)* permite regresar al fragmento de la unión o continuar hacia la secuencia final abb. Los movimientos ε enlazan esos componentes sin leer nuevos caracteres.

3.3. Conversión del AFND a AFD por subconjuntos

El archivo er_afd/afd.py desarrolla la construcción de subconjuntos. Su principio es que un solo estado del AFD puede representar varios estados posibles del AFND. En lugar de mantener opciones diferentes para el mismo símbolo, el sistema reúne los estados alcanzables dentro de un conjunto.

- **A) Cierre épsilon:**

El cierre épsilon de un conjunto está formado por sus estados originales y todos aquellos a los que es posible llegar recorriendo únicamente transiciones ε. La función cierre_epsilon agrega estados a una lista hasta que ya no encuentra nuevos destinos; después ordena el resultado. El primer estado determinista, Q0, representa precisamente el cierre épsilon de q0.

- **B) Movimiento y creación de estados:**

A partir de cada conjunto, la función mover busca los destinos alcanzables cuando se lee un símbolo del alfabeto. Luego se calcula el cierre épsilon de esos destinos. Si el conjunto resultante no ha aparecido antes, el programa crea un nuevo estado Q1, Q2, etc. El proceso continúa mientras existan conjuntos sin revisar.

Un estado del AFD se considera de aceptación cuando incluye algún estado final del AFND. El símbolo ε no se utiliza como carácter de entrada ni forma parte del alfabeto del AFD, porque sus posibles recorridos ya fueron incorporados en los cierres épsilon.

- **C) Ejemplo real del AFD generado:**

En la misma expresión (a|b)*abb, el AFND de 14 estados se convierte en un AFD de cinco estados, Q0 a Q4. La correspondencia de los estados deterministas con los subconjuntos del AFND es:

Estado AFD	Estados AFND representados
Q0	q0, q1, q2, q4, q6, q8
Q1	q1, q2, q3, q4, q5, q6, q8, q9, q10
Q2	q1, q2, q3, q4, q6, q7, q8
Q3	q1, q2, q3, q4, q6, q7, q8, q11, q12
Q4	q1, q2, q3, q4, q6, q7, q8, q13

Como Q4 contiene q13, es el único estado final de este AFD. La tabla determinista que imprime el programa es:

Estado actual	Símbolo	Estado destino
Q0	a	Q1
Q0	b	Q2
Q1	a	Q1
Q1	b	Q3
Q2	a	Q1
Q2	b	Q2
Q3	a	Q1
Q3	b	Q4
Q4	a	Q1
Q4	b	Q2

Cada combinación de estado y símbolo de esta tabla tiene un único destino. Esa propiedad permite recorrer el AFD de forma directa, sin elegir entre varias alternativas en cada carácter.

3.4. Simulación de una cadena de entrada

La función evaluar_cadena, implementada en er_afd/simulador.py, inicia en Q0 y examina cada carácter de izquierda a derecha. Para cada símbolo, busca la transición correspondiente en el AFD, guarda el recorrido y actualiza el estado actual. Si no existe transición válida, la cadena se rechaza. Cuando se consume toda la entrada, la cadena se acepta solamente si el estado alcanzado es final.

Con la ER (a|b)*abb y la cadena abb, el recorrido presentado por el programa es:

```
Q0|a|Q1
Q1|b|Q3
Q3|b|Q4
Cadena aceptada
```

En este caso se llega a Q4, que es de aceptación. Como contraste, una cadena que no termina en abb será rechazada por este patrón. El simulador comprueba que la cadena contenga únicamente caracteres ASCII alfanuméricos o espacios; las entradas con otros símbolos se consideran inválidas antes de simular.

El programa principal devuelve el código 1 cuando una cadena es aceptada, 0 cuando es rechazada y 2 cuando los argumentos son inválidos. El objetivo make run interpreta los códigos 0 y 1 como ejecuciones normales para que Make no señale un error solamente por haberse aceptado la cadena.

3.5. Relación de módulos y archivos consultados

Archivo	Función dentro del proyecto
er_afd/analizador.py	Analiza la sintaxis y produce la estructura AST.
er_afd/afnd.py	Construye el AFND con Thompson y presenta sus transiciones.
er_afd/afd.py	Calcula los cierres ε, conjuntos y transiciones del AFD.
er_afd/simulador.py	Recorre el AFD y decide la aceptación de la cadena.
er_afd/cli.py	Recibe los argumentos, conecta módulos y presenta el resultado.
tests/ y pruebas_precision.py	Verifican las funciones y ejecutan la batería de 36 casos.

Esta organización separa las responsabilidades y permite comprobar cada etapa de forma independiente, desde el reconocimiento de la expresión hasta el resultado final.

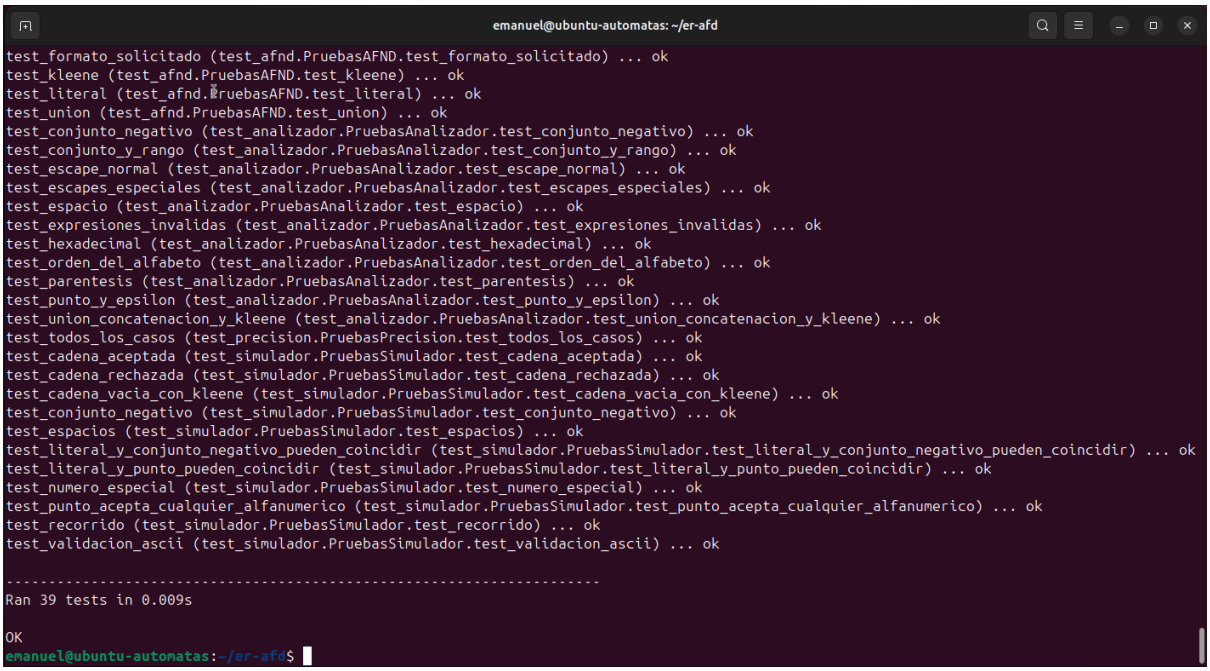
3.6. Evidencias de ejecución

Las figuras siguientes corresponden a capturas originales tomadas durante la ejecución del proyecto en Ubuntu 24.04.5 ARM64, en una máquina virtual UTM sobre un equipo Apple M4. Algunas imágenes muestran recortes de las capturas para destacar la información relevante y facilitar su lectura; no se modificaron los resultados del programa.

Figura 2. Pruebas automáticas

Salida final de los tests, mostrando el número total de pruebas y el resultado OK.

`make test`

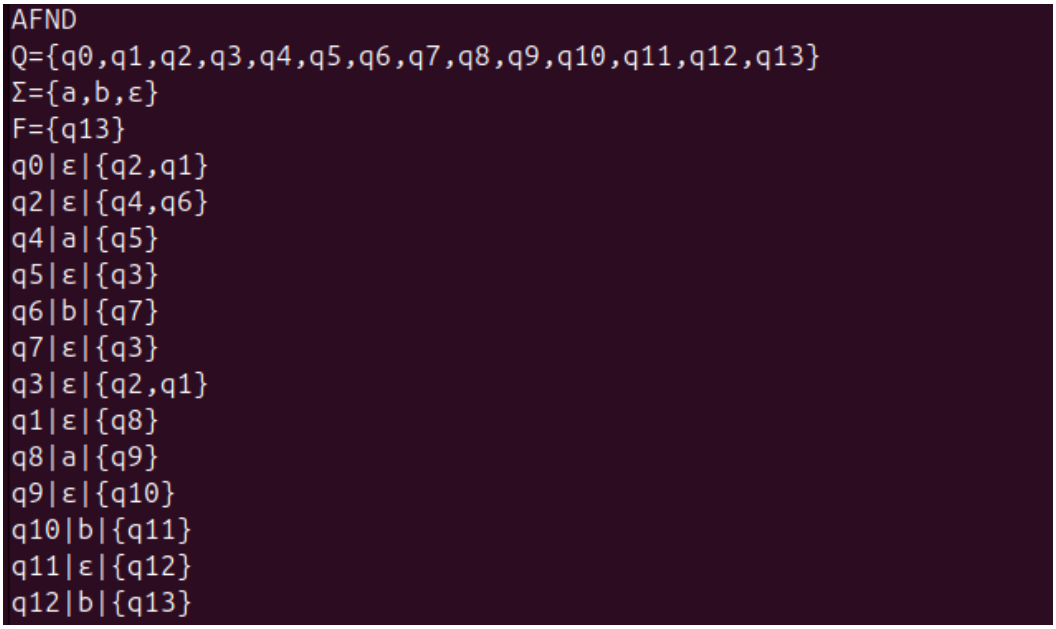


Fuente: captura original de la ejecución del proyecto en Ubunu.

Figura 3. Tabla completa del AFND

Sección AFND con estados, alfabeto, estados finales y todas las transiciones.

`make run ER='(a|b)*abb' CADENA='abb'`



Fuente: captura original de la ejecución del proyecto en Ubuntu.

Figura 4. Tabla completa del AFD

Sección AFD con estados, alfabeto, finales y transiciones deterministas.

```
make run ER='(a|b)*abb' CADENA='abb'
```

```
AFD
Q={Q0,Q1,Q2,Q3,Q4}
Σ={a,b}
F={Q4}
Q0|a|Q1
Q0|b|Q2
Q1|a|Q1
Q1|b|Q3
Q2|a|Q1
Q2|b|Q2
Q3|a|Q1
Q3|b|Q4
Q4|a|Q1
Q4|b|Q2
Q0|a|Q1
Q1|b|Q3
Q3|b|Q4
```

Fuente: captura original de la ejecución del proyecto en Ubuntu.

Figura 5. Ejecución y resultado

Evidencia de la expresión con unión, concatenación y Kleene y del recorrido hasta Cadena aceptada.

```
make run ER='(a|b)*abb' CADENA='abb'
```

```
OK
emanuel@ubuntu-automatas:~/er-afd$ make run ER='(a|b)*abb' CADENA='abb'
AFND
Q={q0,q1,q2,q3,q4,q5,q6,q7,q8,q9,q10,q11,q12,q13}
Σ={a,b,ε}
F={q13}
q0|ε|{q2,q1}
q2|ε|{q4,q6}
q4|a|{q5}
q5|ε|{q3}
q6|b|{q7}
q7|ε|{q3}
q3|ε|{q2,q1}
q1|ε|{q8}
q8|a|{q9}
q9|ε|{q10}
q10|b|{q11}
q11|ε|{q12}
q12|b|{q13}
AFD
```



Fuente: captura original de la ejecución del proyecto en Ubuntu).

3.7. Consideraciones técnicas y límites de la ejecución

Para interpretar correctamente el resultado, debe diferenciarse entre una expresión regular inválida y una cadena rechazada. Si falla la sintaxis de la ER o la validación de los caracteres permitidos, el programa termina con un mensaje de error; en cambio, una cadena rechazada es una entrada válida que no conduce a un estado de aceptación. Esta distinción evita confundir errores de uso con el funcionamiento normal del autómata.

La simulación está diseñada para cadenas formadas por letras y dígitos ASCII, además de espacios. Aunque el analizador reconoce escapes y símbolos de la sintaxis de ER, el tratamiento de la cadena está condicionado por esa validación. Del mismo modo, los conjuntos negativos y el comodín se procesan mediante comparaciones específicas del programa; por eso, para comprobar su comportamiento debe revisarse la ejecución real de los casos disponibles y no asumir que el sistema implementa todas las características de los motores profesionales de expresiones regulares.

Para defender el proyecto es útil explicar el orden exacto de las operaciones: el parser entrega el AST; Thompson produce el AFND con transiciones ε; el cierre épsilon forma los conjuntos iniciales; las transiciones por símbolo completan el AFD, y el simulador recorre estos estados hasta aceptar o rechazar. Las tablas reales del ejemplo de la sección 5 permiten ilustrar ese proceso durante la exposición.

4. Informe de Verificación: Casos de Prueba y Resultados

4.1. Propósito

Verificar, sin modificar el código, que el programa convierte expresiones regulares en un AFND y un AFD, y que decide correctamente si una cadena es aceptada o rechazada. Este informe reúne las pruebas automáticas, la batería de precisión, ejemplos con su recorrido y una tabla de casos por categoría.

4.2. Resumen de resultados

Verificación	Resultado
make test (pruebas unitarias)	39 pruebas · OK
pruebas_precision.py	36 de 36 · 100.00 %
Tabla ampliada (sección 4): casos	48
Tabla ampliada: aprobados	48
Tabla ampliada: fallidos	0
Precisión de la tabla ampliada	100.00 %

Las pruebas automáticas, la batería de precisión y los ejemplos con recorrido se ejecutaron en un equipo con Ubuntu (máquina virtual VirtualBox) el 5 de noviembre de 2026; las capturas de la sección 4.3 respaldan esos resultados. Los 48 casos de la tabla de la sección 4.4 coinciden con el resultado esperado definido en el repositorio.

4.3. Evidencias de ejecución

Esta sección documenta, con capturas originales de la terminal de Ubuntu, las verificaciones principales realizadas sobre el programa: la suite de pruebas automáticas, la batería de precisión y cuatro ejecuciones con su recorrido (cadena aceptada, cadena rechazada, rango con escape especial y conjunto negativo). Todos los comandos se ejecutaron desde el directorio ~/er-afd. En cada ejecución el programa imprime, en este orden, la tabla del AFND, la tabla del AFD, el recorrido paso a paso de la cadena y el veredicto.

- **1) Pruebas automáticas**

```
make test
```

El objetivo make test invoca. venv/bin/python -m unittest discover -s tests -v. El módulo estándar unittest descubre y ejecuta todos los archivos de la carpeta tests/ (analizador, AFND, AFD, simulador y precisión) en modo detallado, de modo que cada prueba aparece con su nombre y su estado. Como se observa en la

Figura 1, las 39 pruebas terminan en «ok» y la ejecución finaliza con «Ran 39 tests in 0.015s» y «OK», sin fallos ni errores.

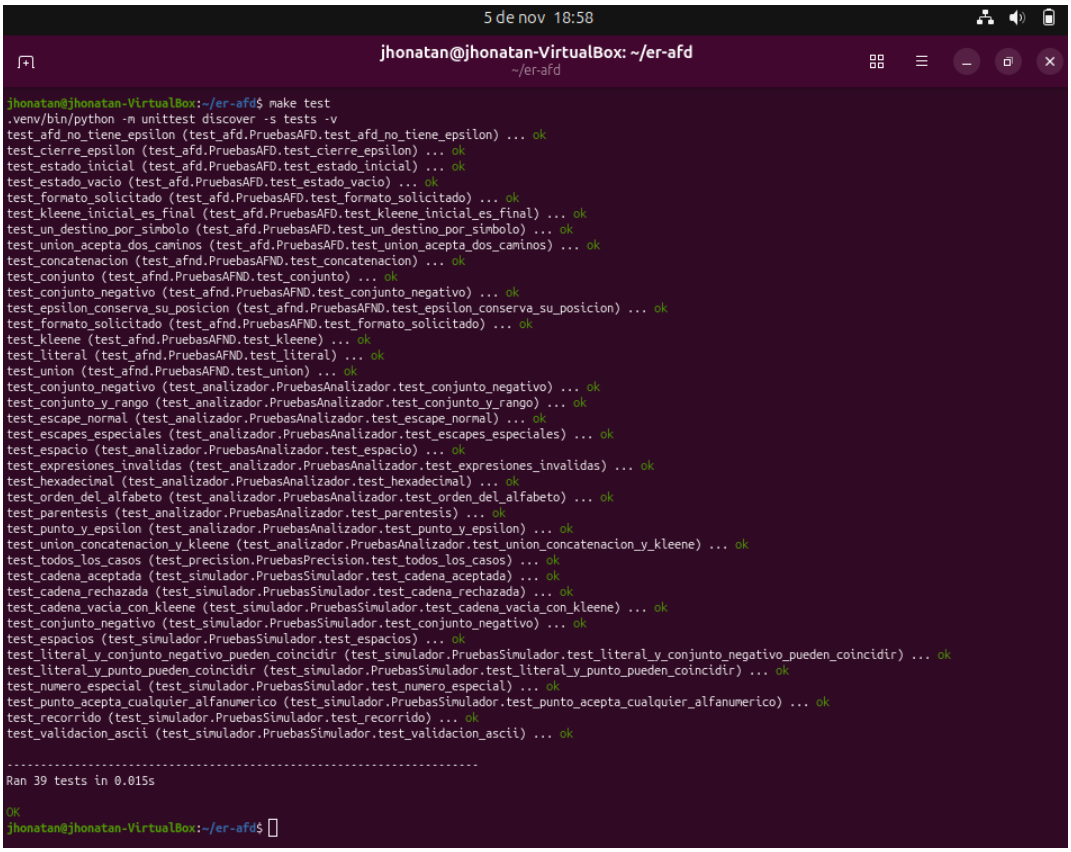


Figura 1. Pruebas automáticas (make test)

Listado de las 39 pruebas unitarias con estado «ok» y resultado final «OK».

Fuente: captura original de la ejecución del proyecto en Ubuntu (elaboración propia).

- **2) Batería de precisión**

```
.venv/bin/python pruebas_precision.py
```

El script pruebas_precision.py recorre la lista de 36 casos definida en el repositorio. Para cada par (expresión regular, cadena) construye el árbol sintáctico, el AFND y el AFD, evalúa la cadena y compara el veredicto con el resultado esperado, imprimiendo el número de caso, la ER, la cadena, lo esperado, lo obtenido y el estado («OK» o «FALLO»). La Figura 2 muestra los 36 casos en «OK» y el cierre «Casos correctos: 36 de 36» con «Precisión: 100.00%», lo que supera el umbral de 95 % de la rúbrica de detección precisa.

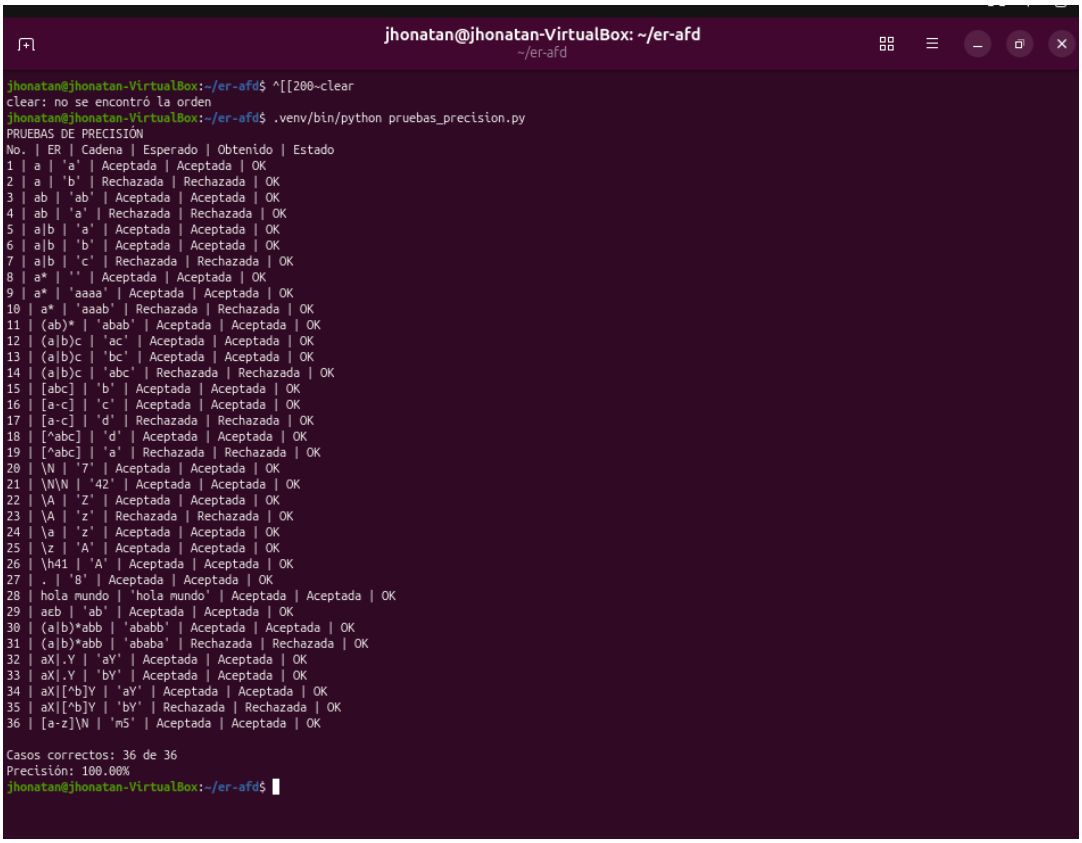


Figura 2. Batería de precisión

Salida de pruebas_precision.py: 36 de 36 casos correctos, precisión de 100.00 %.
Fuente: captura original de la ejecución del proyecto en Ubuntu (elaboración propia).

- **3) Cadena aceptada con su recorrido**

`make run`

Sin argumentos, make run usa los valores por defecto del Makefile: ER='(a|b)*abb' y CADENA='abb'. Esta expresión reconoce las cadenas formadas por a y b que terminan en «abb». El AFND, construido con el algoritmo de Thompson, tiene 14 estados (q0 a q13), alfabeto Σ={a,b,ε} y estado de aceptación F={q13}. Tras la construcción de subconjuntos, el AFD queda con 5 estados (Q0 a Q4), Σ={a,b} y F={Q4}. El recorrido de la cadena es Q0|a|Q1, Q1|b|Q3 y Q3|b|Q4; como Q4 es un estado de aceptación, el programa responde «Cadena aceptada» (código de retorno 1).

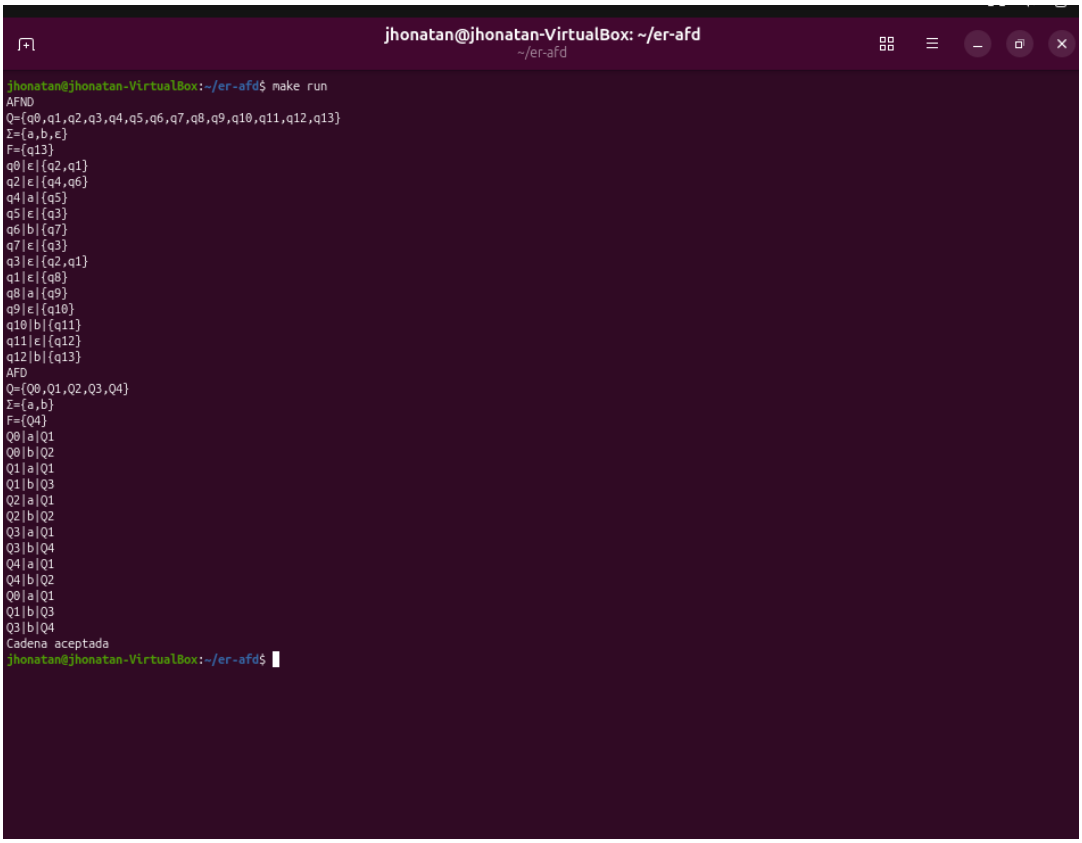


Figura 3. Cadena aceptada con su recorrido

*Ejecución de make run con (a|b)*abb y abb: recorrido Q0→Q1→Q3→Q4 y resultado «Cadena aceptada».*

Fuente: captura original de la ejecución del proyecto en Ubuntu (elaboración propia).

- **4) Cadena rechazada con su recorrido**

```
make run ER='(a|b)*abb' CADENA='aba'
```

Con la misma expresión y la cadena aba, el AFND y el AFD son idénticos a los del caso anterior. El recorrido es Q0|a|Q1, Q1|b|Q3 y Q3|a|Q1: la cadena se consume por completo, pero el autómata termina en Q1, que no pertenece a F={Q4}. El programa responde «Cadena rechazada» (código de retorno 0). Este caso demuestra que el simulador no solo valida que existan transiciones, sino que exige terminar en un estado de aceptación.

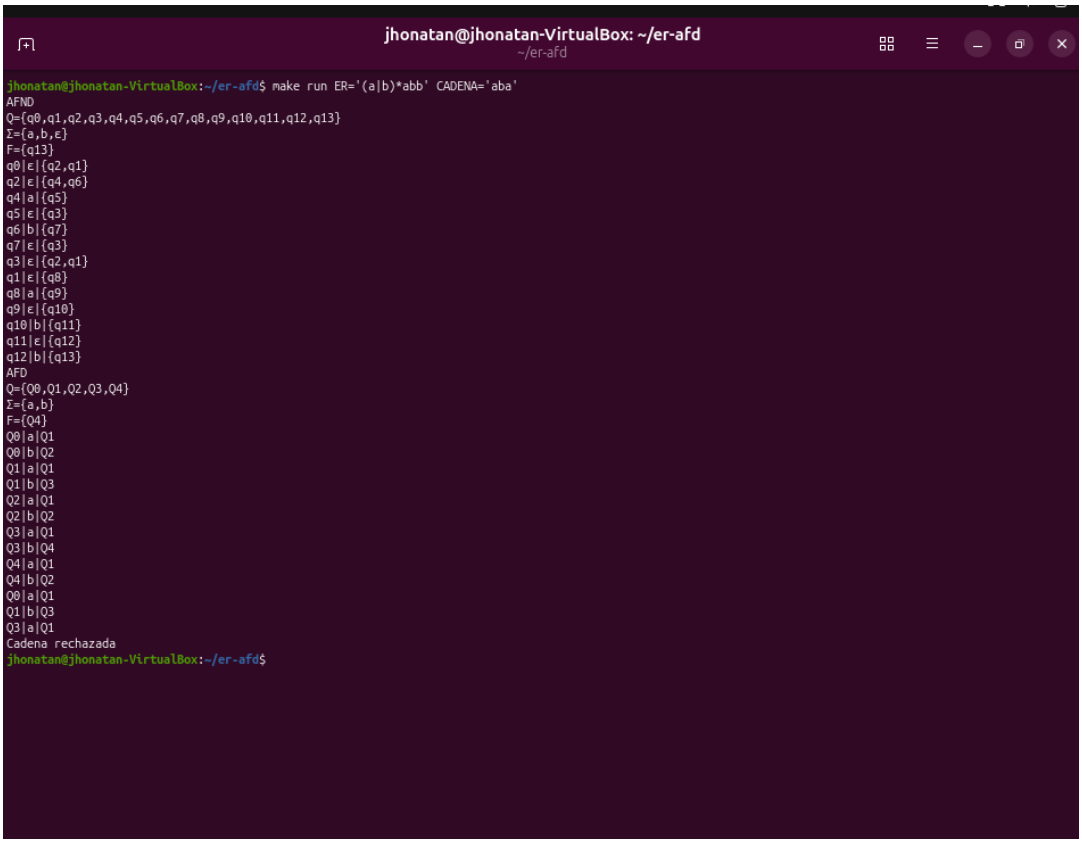


Figura 4. Cadena rechazada con su recorrido

*Ejecución con (a|b)*abb y aba: el recorrido termina en Q1 y el resultado es «Cadena rechazada». Fuente: captura original de la ejecución del proyecto en Ubuntu (elaboración propia).*

- **5) Rango con escape especial**

```
make run ER='[a-z]\N' CADENA='m5'
```

La expresión [a-z]\N concatena un rango de letras minúsculas con la secuencia especial \N, equivalente a [0-9]. El AFND resultante tiene 4 estados (q0 a q3), F={q3} y un alfabeto de 37 símbolos: las 26 letras, los 10 dígitos y ε; la unión de ambos operandos se materializa en la transición q1|ε|{q2}. El AFD tiene 4 estados (Q0 a Q3), F={Q3} y 36 símbolos. Q0 pasa a Q1 con una letra, Q1 pasa a Q3 con un dígito y Q2 actúa como estado sumidero, al que se llega con cualquier símbolo fuera de lugar y del que no se sale. El recorrido de «m5» es Q0|m|Q1 y Q1|5|Q3, por lo que la cadena es aceptada.

Debido a la longitud de la salida (cada estado tiene una transición por cada uno de los 36 símbolos), la ejecución se documentó en cinco capturas consecutivas, Figuras 5 a 9, que se solapan parcialmente para no omitir ninguna línea.

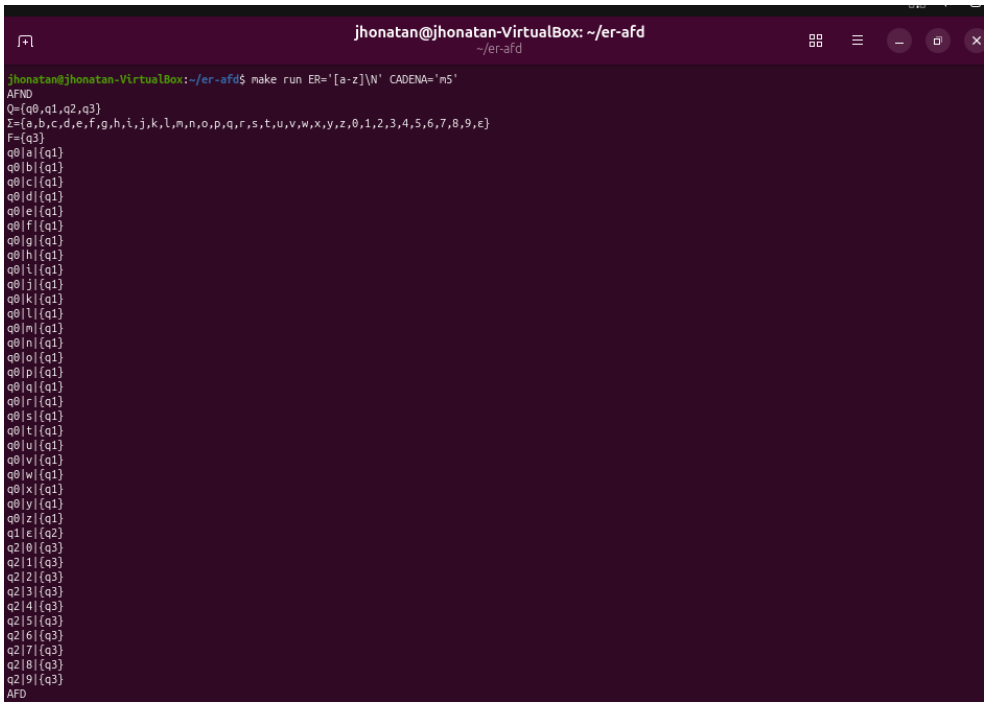


Figura 5. Rango con escape especial: comando y tabla del AFND (1 de 5)

Comando ejecutado y AFND con Σ de 36 símbolos más ε, F={q3} y la transición q1|ε|{q2}.

Fuente: captura original de la ejecución del proyecto en Ubuntu (elaboración propia).

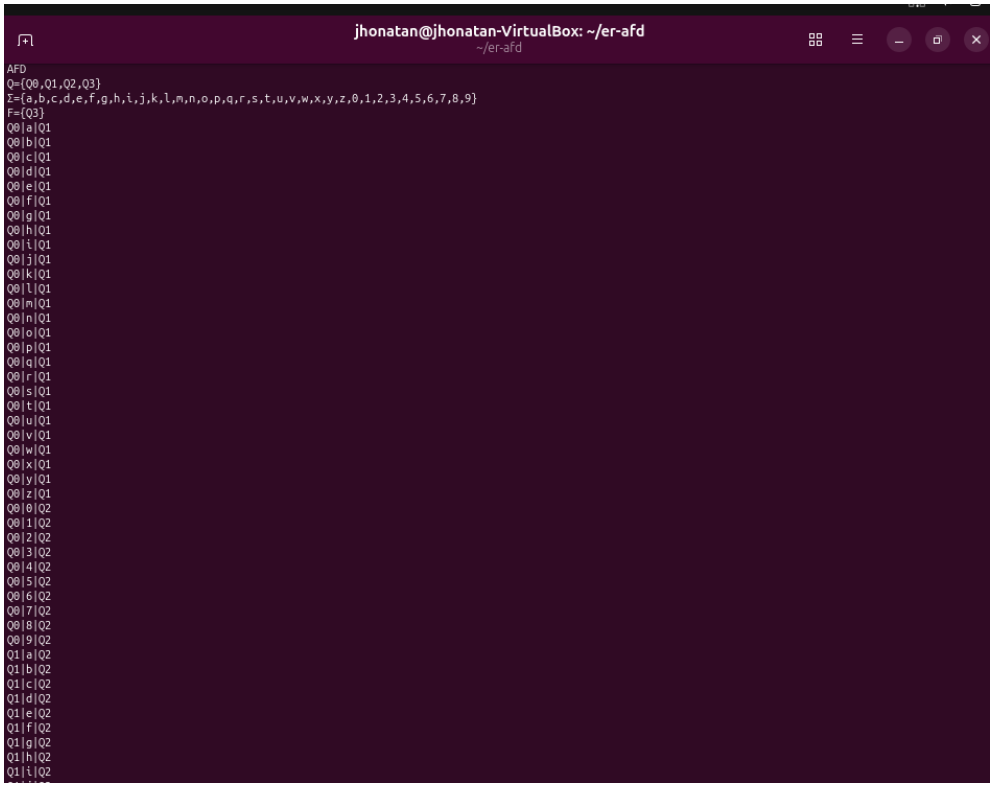


Figura 6. Rango con escape especial: inicio de la tabla del AFD (2 de 5)

AFD con Q={Q0,Q1,Q2,Q3} y F={Q3}; Q0 va a Q1 con letras y a Q2 con dígitos, y comienzan las transiciones de Q1.

Fuente: captura original de la ejecución del proyecto en Ubuntu (elaboración propia).

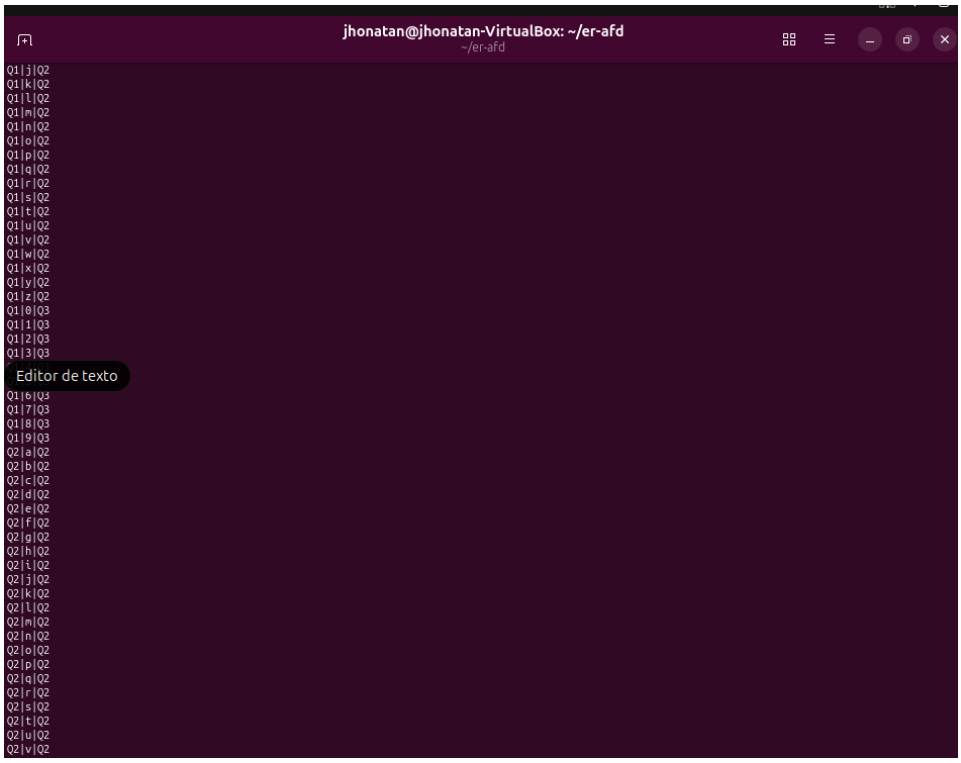


Figura 7. Rango con escape especial: continuación del AFD (3 de 5)

Transiciones de Q1 y de Q2; Q2 permanece en Q2 con cualquier símbolo (estado sumidero).

Fuente: captura original de la ejecución del proyecto en Ubuntu (elaboración propia).

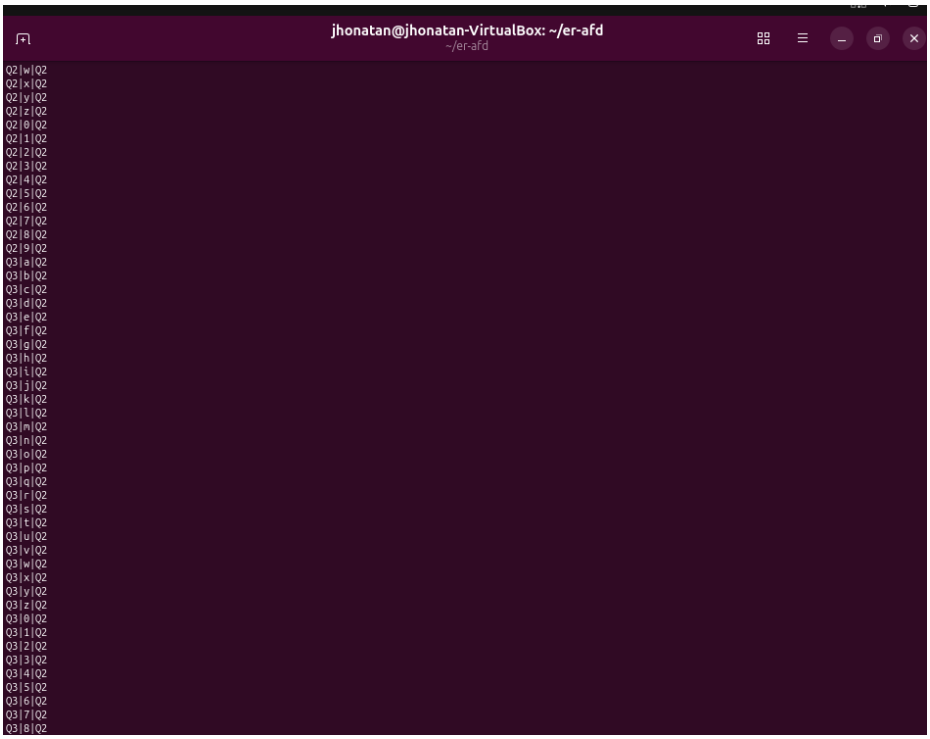


Figura 8. Rango con escape especial: continuación del AFD (4 de 5)

Continuación de las transiciones de Q2 y comienzo de las de Q3.

Fuente: captura original de la ejecución del proyecto en Ubuntu (elaboración propia).

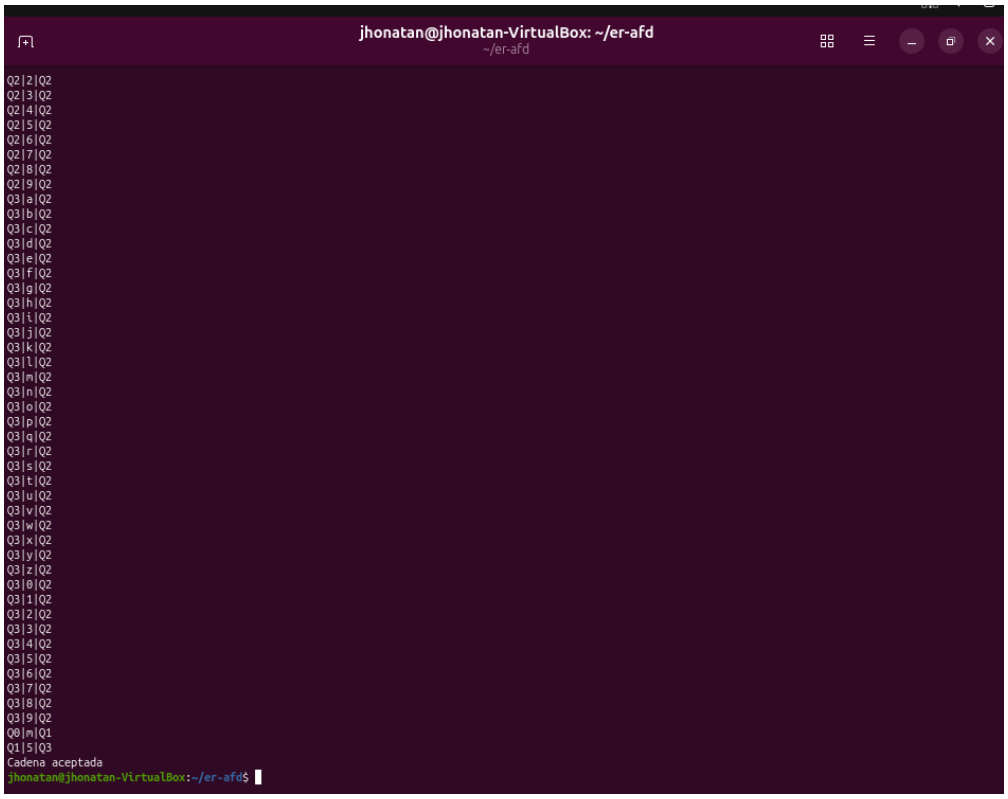


Figura 9. Rango con escape especial: final, recorrido y veredicto (5 de 5)

Transiciones de Q3, recorrido Q0|m|Q1 y Q1|5|Q3 y resultado «Cadena aceptada».
Fuente: captura original de la ejecución del proyecto en Ubuntu (elaboración propia).

- **6) Conjunto negativo**

```
make run ER='[^abc]' CADENA='d'
```

El conjunto negativo [^abc] reconoce cualquier símbolo distinto de a, b y c. En las tablas, el programa representa el conjunto como un único símbolo del alfabeto: el AFND tiene 2 estados (q0 y q1), Σ={[^abc],ε} y F={q1}; el AFD tiene 3 estados (Q0 a Q2), Σ={[^abc]} y F={Q1}, donde Q2 es el estado sumidero. La simulación evalúa el carácter real contra el conjunto: «d» no pertenece a {a,b,c}, por lo que el recorrido es Q0|d|Q1 y la cadena es aceptada.

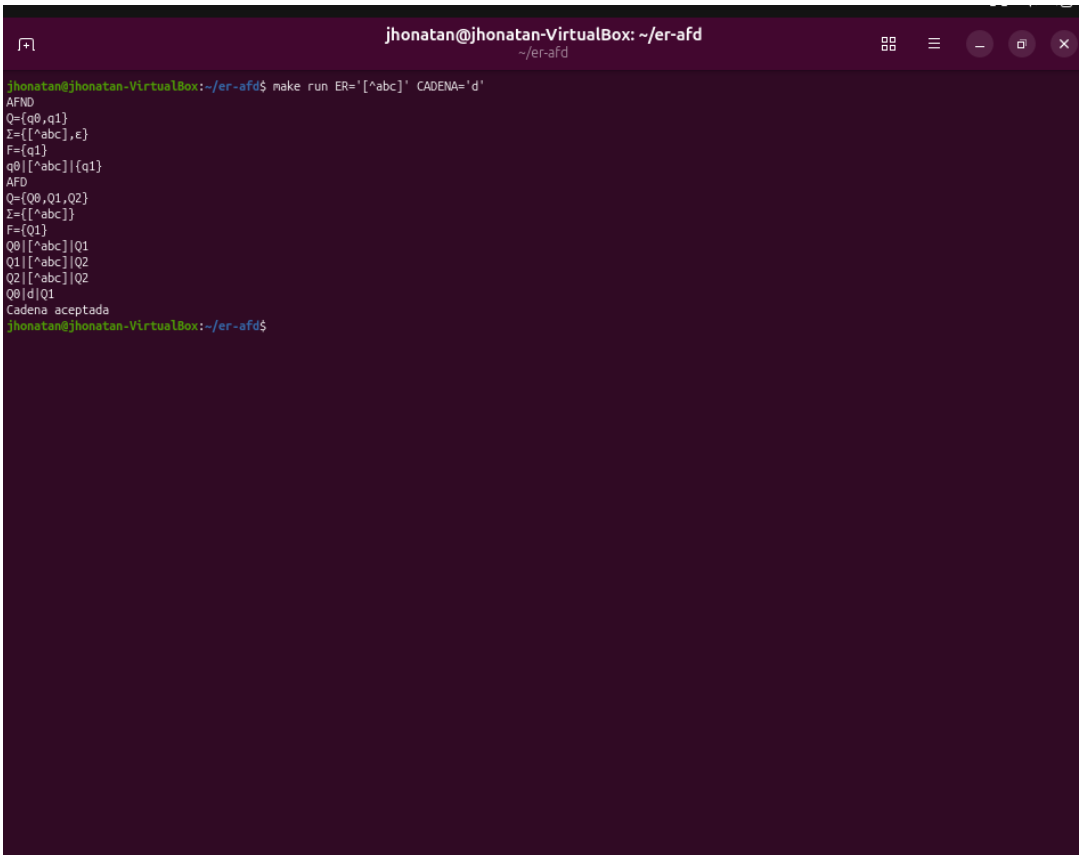


Figura 10. Conjunto negativo [^abc] con la cadena d

AFND, AFD y recorrido Q0|d|Q1 con resultado «Cadena aceptada».

Fuente: captura original de la ejecución del proyecto en Ubuntu (elaboración propia).

4.4. Tabla de casos de pruebas

N.º de Prueba	Expresión regular	Cadena	Resultado esperado	Resultado obtenido	Aprobado / Fallido
Símbolo simple					
1	a	a	Cadena aceptada	Cadena aceptada	Aprobado
2	a	b	Cadena rechazada	Cadena rechazada	Aprobado
Concatenación					
3	ab	ab	Cadena aceptada	Cadena aceptada	Aprobado
4	ab	a	Cadena rechazada	Cadena rechazada	Aprobado

Unión					
5	a|b	a	Cadena aceptada	Cadena aceptada	Aprobado
6	a|b	b	Cadena aceptada	Cadena aceptada	Aprobado
7	a|b	c	Cadena rechazada	Cadena rechazada	Aprobado
Cerradura de Kleene					
8	a*	(cadena vacía)	Cadena aceptada	Cadena aceptada	Aprobado
9	a*	aaaa	Cadena aceptada	Cadena aceptada	Aprobado
10	a*	aaab	Cadena rechazada	Cadena rechazada	Aprobado
11	(ab)*	abab	Cadena aceptada	Cadena aceptada	Aprobado
Paréntesis					
12	(a|b)c	ac	Cadena aceptada	Cadena aceptada	Aprobado
13	(a|b)c	bc	Cadena aceptada	Cadena aceptada	Aprobado
14	(a|b)c	abc	Cadena rechazada	Cadena rechazada	Aprobado
15	(a|b)*abb	ababb	Cadena aceptada	Cadena aceptada	Aprobado
16	(a|b)*abb	ababa	Cadena rechazada	Cadena rechazada	Aprobado
Punto					
17	.	8	Cadena aceptada	Cadena aceptada	Aprobado
Épsilon					
18	aεb	ab	Cadena aceptada	Cadena aceptada	Aprobado
Conjuntos					
19	[abc]	b	Cadena aceptada	Cadena aceptada	Aprobado
Conjuntos negativos					
20	[^abc]	d	Cadena aceptada	Cadena aceptada	Aprobado
21	[^abc]	a	Cadena rechazada	Cadena rechazada	Aprobado
Rangos					

22	[a-c]	c	Cadena aceptada	Cadena aceptada	Aprobado
23	[a-c]	d	Cadena rechazada	Cadena rechazada	Aprobado
24	[a-z]\N	m5	Cadena aceptada	Cadena aceptada	Aprobado
Escapes especiales					
25	\N	7	Cadena aceptada	Cadena aceptada	Aprobado
26	\N\N	42	Cadena aceptada	Cadena aceptada	Aprobado
27	\A	Z	Cadena aceptada	Cadena aceptada	Aprobado
28	\A	z	Cadena rechazada	Cadena rechazada	Aprobado
29	\a	z	Cadena aceptada	Cadena aceptada	Aprobado
30	\z	A	Cadena aceptada	Cadena aceptada	Aprobado
31	\h41	A	Cadena aceptada	Cadena aceptada	Aprobado
Espacios					
32	hola mundo	hola mundo	Cadena aceptada	Cadena aceptada	Aprobado
Combinaciones (literal, punto y conjunto negativo)					
33	aX|.Y	aY	Cadena aceptada	Cadena aceptada	Aprobado
34	aX|.Y	bY	Cadena aceptada	Cadena aceptada	Aprobado
35	aX|[^b]Y	aY	Cadena aceptada	Cadena aceptada	Aprobado
36	aX|[^b]Y	bY	Cadena rechazada	Cadena rechazada	Aprobado
Expresiones inválidas					
37	(vacía)	a	ER inválida	ER inválida	Aprobado
38	|a	a	ER inválida	ER inválida	Aprobado
39	a|	a	ER inválida	ER inválida	Aprobado
40	a||b	a	ER inválida	ER inválida	Aprobado
41	()	a	ER inválida	ER inválida	Aprobado
42	(ab	a	ER inválida	ER inválida	Aprobado

43	ab)	a	ER inválida	ER inválida	Aprobado
44	[]	a	ER inválida	ER inválida	Aprobado
45	[a	a	ER inválida	ER inválida	Aprobado
46	[z-a]	a	ER inválida	ER inválida	Aprobado
47	\	a	ER inválida	ER inválida	Aprobado
48	\h4G	a	ER inválida	ER inválida	Aprobado

- **Resultados por categoría**

La tabla y el gráfico resumen cuántos casos de cada categoría fueron aprobados. Todas las categorías aprueban al 100%.

Categoría	**Casos**	**Aprobados**	**Fallidos**
Símbolo simple	2	2	0
Concatenación	2	2	0
Unión	3	3	0
Cerradura de Kleene	4	4	0
Paréntesis	5	5	0
Punto	1	1	0
Épsilon	1	1	0
Conjuntos	1	1	0
Conjuntos negativos	2	2	0
Rangos	3	3	0
Escapes especiales	7	7	0
Espacios	1	1	0
Combinaciones (literal, punto y conjunto negativo)	4	4	0
Expresiones inválidas	12	12	0
Total	**48**	**48**	**0**
Precisión		**100.00%**	

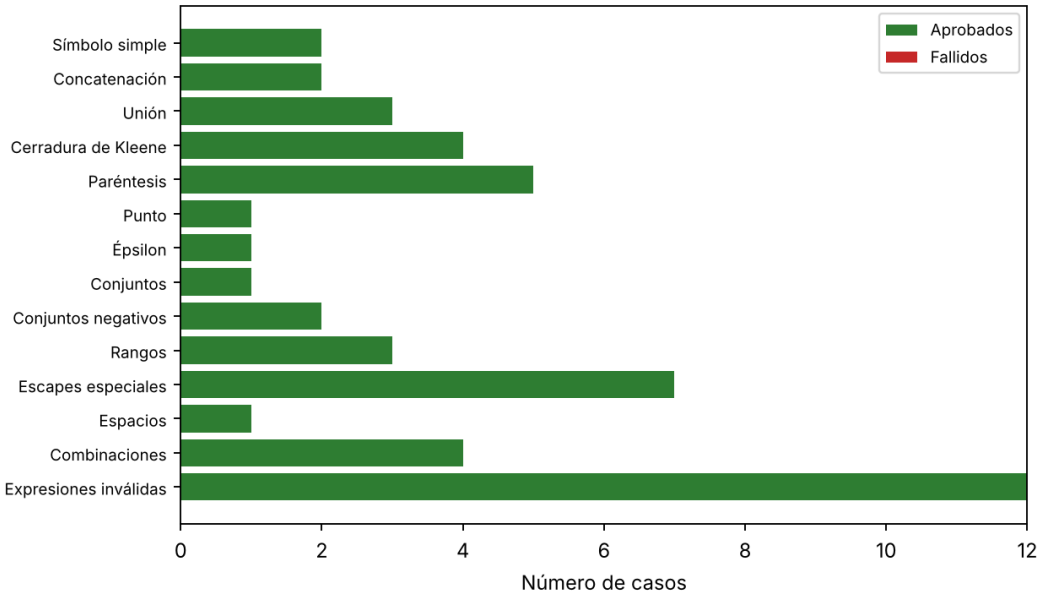


Figura 11. Casos aprobados y fallidos por categoría

Fuente: elaboración propia a partir de la tabla de casos de prueba (sección 4).

- **Análisis técnico por categoría**

La tabla de casos no es una lista arbitraria: cada categoría pone a prueba una característica concreta del analizador, de la construcción de Thompson, de la construcción de subconjuntos o del simulador. La siguiente tabla resume qué valida cada grupo de casos.

Categoría	Casos	¿Qué se verifica?
Símbolo simple	1–2	El AFND mínimo de un símbolo acepta únicamente ese símbolo y rechaza cualquier otro.
Concatenación	3–4	La yuxtaposición implícita: ab acepta «ab» y rechaza el prefijo incompleto «a».
Unión	5–7	El operador |: acepta cualquiera de las alternativas y rechaza un símbolo ajeno.
Cerradura de Kleene	8–11	Cero o más repeticiones: acepta la cadena vacía, repeticiones largas y grupos como (ab)*, y rechaza una repetición interrumpida (aaab).

Categoría	Casos	¿Qué se verifica?
Paréntesis	12–16	La precedencia: (a|b)c no acepta «abc», y (a|b)*abb acepta «ababb» y rechaza «ababa».
Punto	17	El comodín. acepta cualquier símbolo del alfabeto de entrada.
Épsilon	18	ε se conserva en su posición dentro de la ER: aεb acepta «ab».
Conjuntos	19	[abc] equivale a a|b|c.
Conjuntos negativos	20–21	[^abc] acepta símbolos fuera del conjunto («d») y rechaza los incluidos («a»).
Rangos	22–24	[a-c] en los extremos y fuera del rango, y la combinación [a-z]\N.
Escapes especiales	25–31	\N (dígito), \N\N, \A (mayúscula: acepta Z, rechaza z), \a (minúscula), \z (cualquier letra) y \h41 (código hexadecimal de la letra A).
Espacios	32	Una ER con espacio literal, entregada entre comillas, reconoce «hola mundo».
Combinaciones	33–36	aX|.Y y aX|[^b]Y: verifican que, cuando un literal, el punto y un conjunto negativo coinciden o se distinguen, el simulador elige la transición correcta.
Expresiones inválidas	37–48	Doce errores sintácticos: ER vacía, | al inicio, | al final, ||, (), paréntesis sin cerrar o sin abrir, [], conjunto sin cerrar, rango invertido [z-a], \ al final y \h con dígito hexadecimal inválido. Todos producen «ER inválida» con causa y posición (código 2).

4.5. Observaciones

- Con el conjunto negativo [^abc], las tablas del AFND y del AFD muestran «[^abc]» como un único símbolo del alfabeto. La simulación es correcta: acepta «d» y rechaza «a».
- make run siempre devuelve código 0 a make. Para ver el código real (1 aceptada, 0 rechazada, 2 inválida) se ejecuta .venv/bin/python -m er_afd <ER> <cadena> y luego echo $?.
- Las expresiones inválidas muestran «ER inválida», la causa y la posición del error.
- La tabla del AFD para [a-z]\N es extensa porque cada estado define una transición por cada símbolo del alfabeto (36); por ello su evidencia se presenta en cinco capturas consecutivas.
- Sin incidencias: todos los casos tomados del repositorio coincidieron con el resultado esperado.

4.6. Consideraciones técnicas y límites de la ejecución

Para interpretar los resultados conviene distinguir tres situaciones. Una cadena rechazada ocurre cuando el autómata termina en un estado que no es de aceptación (código 0). Una expresión inválida ocurre cuando el análisis sintáctico falla y el programa muestra «ER inválida» con la causa y la posición (código 2). Una cadena inválida ocurre cuando la cadena contiene caracteres no permitidos (código 2).

El programa solo admite cadenas alfanuméricas ASCII y espacios, tal como pide el enunciado; cualquier otro carácter en la cadena produce «Cadena inválida» (código 2). Por eso todas las cadenas de prueba usan solo letras, dígitos y espacios.

El código de retorno que pide el enunciado (1 aceptada, 0 rechazada) no se observa con make run, porque el Makefile lo convierte en 0. Se verifica ejecutando el programa directamente y consultando $?.

Los espacios se prueban escapados (hola\ mundo) o escritos tal cual dentro de comillas, como en make run ER='hola mundo'. Los resultados de la tabla dependen de la versión del repositorio evaluada y deben repetirse si el código cambia.

4.7. Conclusión técnica

De 48 casos, 48 fueron aprobados (100.00 %) y 0 fallidos. Las 39 pruebas unitarias y la batería de 36 casos (100.00 %) pasaron sin errores. Las pruebas cubren concatenación, unión, cerradura de Kleene, paréntesis, punto, épsilon, conjuntos, conjuntos negativos, rangos, escapes especiales, espacios y expresiones inválidas, con cadenas aceptadas y rechazadas. El programa construye el AFND y el AFD con el formato pedido, muestra el recorrido paso a paso y señala la causa y la posición de cada expresión inválida.

5. Manual de usuario: Guía rápida de instalación y ejecución

El presente manual tiene como finalidad proporcionar una guía rápida para la instalación, configuración y utilización del sistema ER-AFD, desarrollado en Python para convertir expresiones regulares en autómatas finitos y evaluar cadenas de texto. La herramienta permite realizar el proceso de conversión de una expresión regular (ER) a un Autómata Finito No Determinista (AFND), utilizando el algoritmo de Thompson, y posteriormente transformarlo en un Autómata Finito Determinista (AFD), mediante el método de construcción de subconjuntos.

Finalmente, permite evaluar una cadena de entrada, mostrando el recorrido de los estados y determinando si la cadena es aceptada o rechazada por el autómata generado.

5.1. Requisitos

Para instalar y ejecutar correctamente el proyecto se necesitan los siguientes recursos:

- Sistema operativo: Linux de 64 bits (Ubuntu).
- Lenguaje de programación: Python 3.10 o superior.
- Herramienta de construcción: GNU Make.
- Sistema de control de versiones: Git.
- Entorno virtual: Python venv.
- Conexión a Internet para la instalación inicial de los componentes de construcción.
- Permisos de usuario regular para ejecutar el programa.

No es necesario ejecutar el sistema como administrador. Los permisos de superusuario únicamente se utilizan cuando se requiere instalar herramientas o paquetes del sistema operativo.



Figura 1. Captura real de la terminal de Ubuntu verificando --version

5.2. Descargar el repositorio

Para obtener el código fuente del proyecto se debe abrir una terminal de Ubuntu y ejecutar el siguiente comando:

```
git clone URL_DEL_REPOSITORIO
```

Se debe sustituir URL_DEL_REPOSITORIO por la dirección correspondiente al repositorio alojado en GitHub.

El comando descargará los archivos y directorios necesarios para ejecutar el sistema en el equipo local.

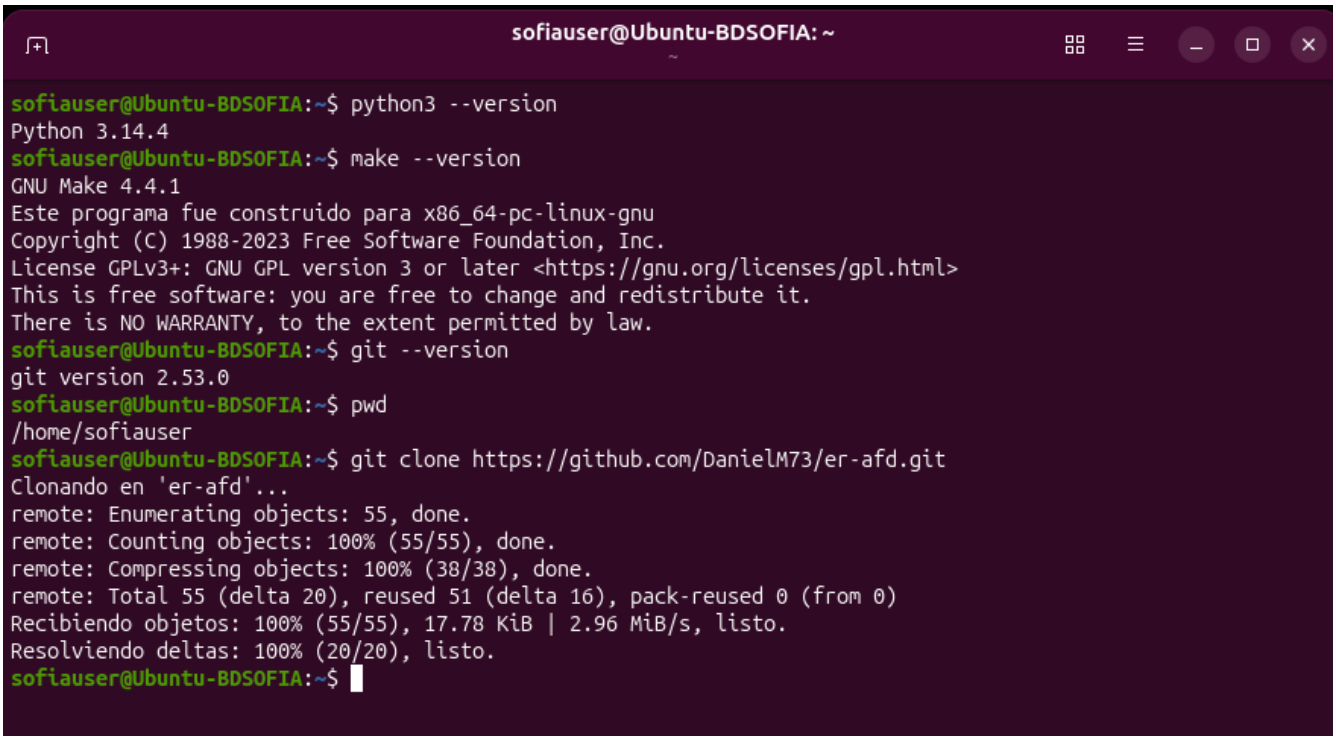


Figura 2. Captura real de la terminal de Ubuntu clonando repositorio

5.3. Ingreso a la carpeta del proyecto

Una vez finalizada la descarga, se debe ingresar al directorio del proyecto mediante:

```
cd er-afd
```

Para verificar que se encuentra en la ubicación correcta, ejecutar:

```
ls
```

Se visualizarán los archivos principales, entre ellos el archivo Makefile, encargado de automatizar las tareas de configuración, construcción, ejecución, pruebas y limpieza.

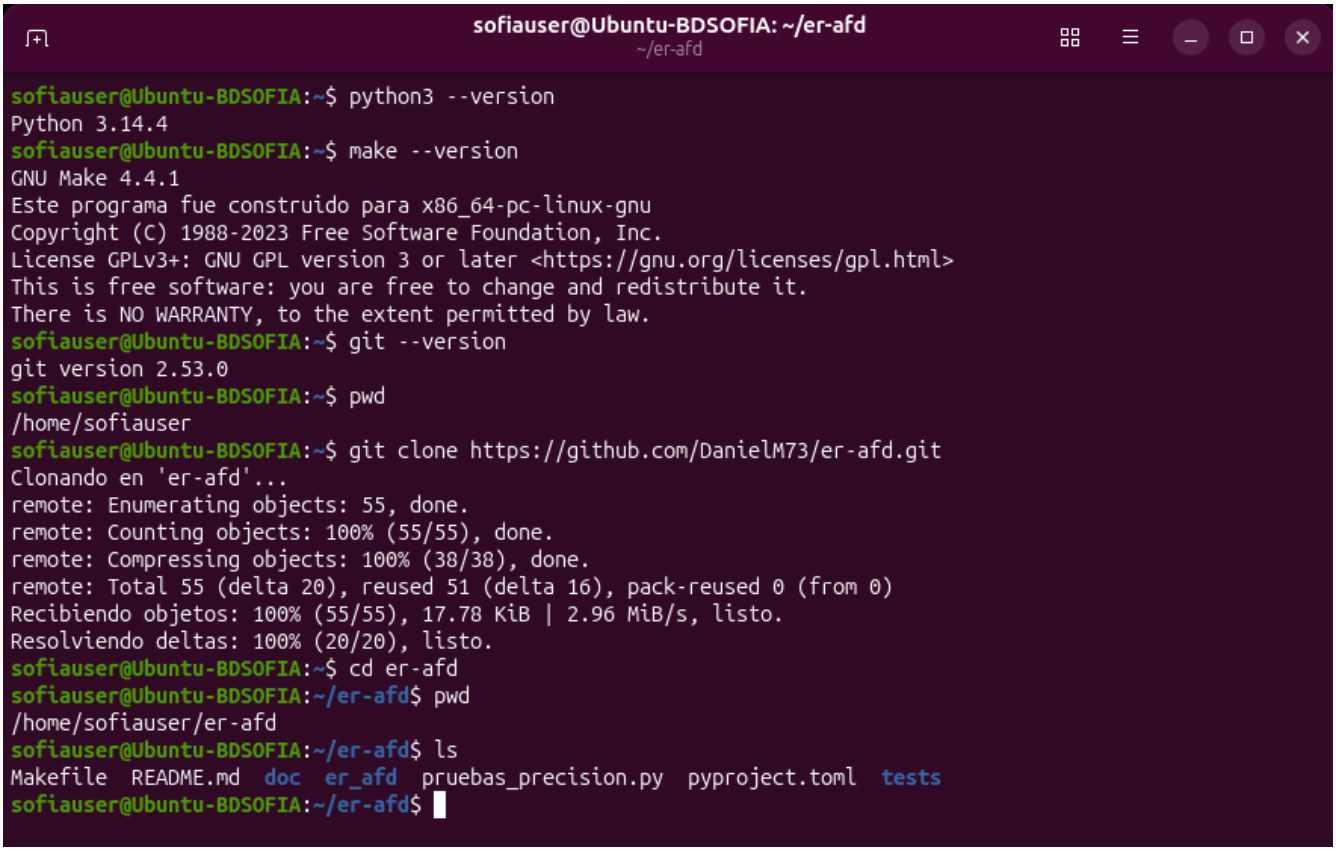


Figura 3. Captura real de la terminal de Ubuntu proyectando repositorio

5.4. Configuración del entorno – make config

El comando:

```
make config
```

permite preparar el entorno de ejecución del proyecto mediante la creación de un entorno virtual de Python, identificado como .venv.

Este entorno mantiene aislados los componentes y herramientas utilizados por el proyecto, evitando interferencias con la instalación general de Python del sistema operativo.

La ejecución correcta del comando genera el directorio .venv dentro de la carpeta principal del proyecto.

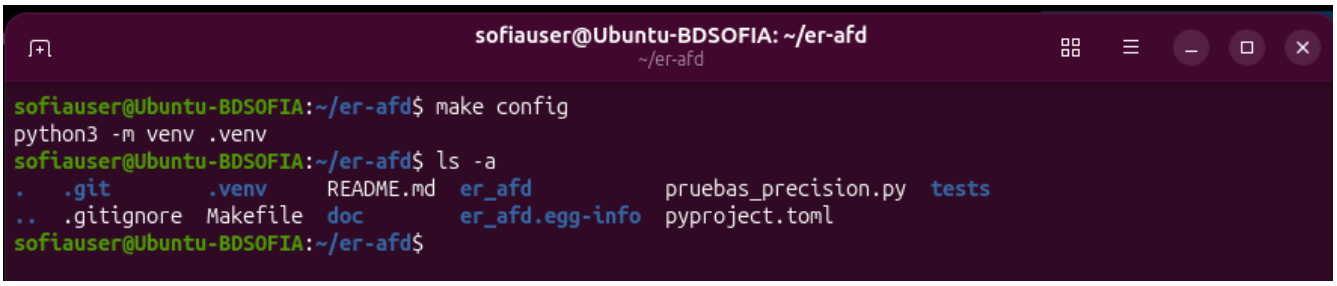


Figura 4. Captura real de la terminal de Ubuntu proyectando directorio .venv

5.5. Construcción del proyecto – make build

Para preparar e instalar el proyecto se utiliza:

`make build`

Este comando realiza las siguientes operaciones:

1. Ejecuta la configuración del entorno virtual.
2. Instala el proyecto en modo editable mediante pip install -e ..
3. Verifica la compilación de los módulos de Python mediante compileall.

Al finalizar correctamente, el proyecto queda preparado para su ejecución desde la terminal y dispone de los componentes necesarios para utilizar los comandos definidos en el Makefile.

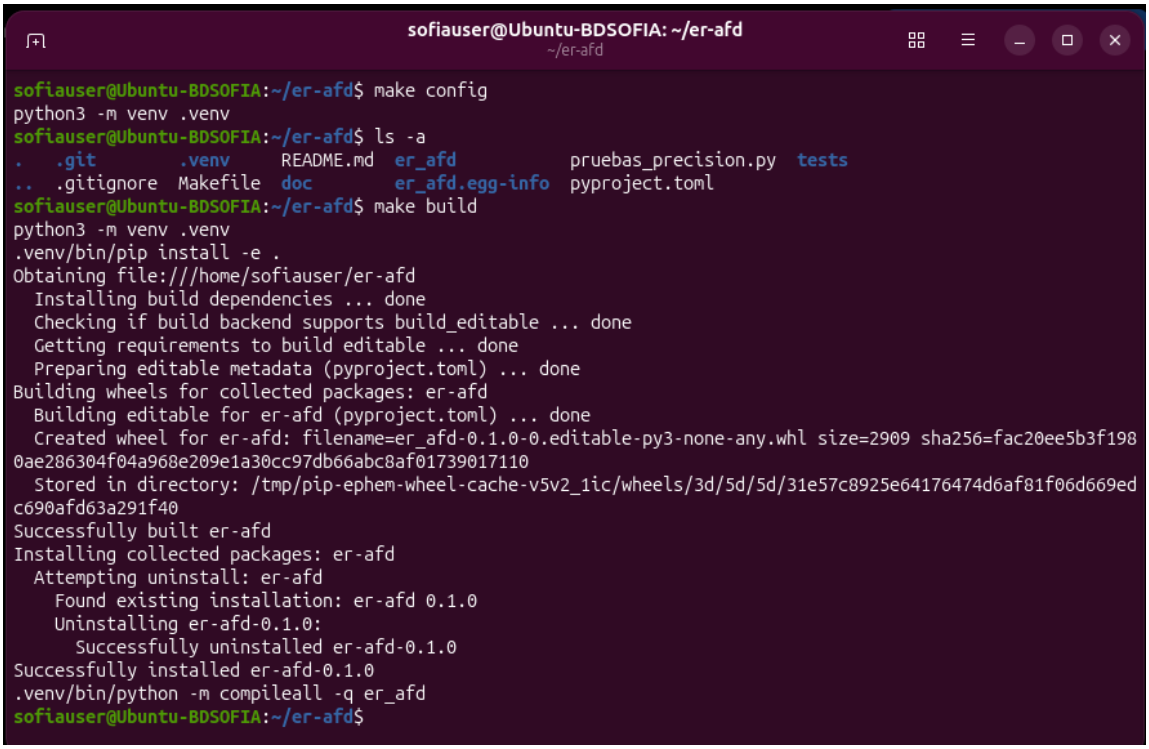


Figura 5. Captura real de la terminal de Ubuntu proyectando make build

5.6. Ejecución del programa – make run

Para ejecutar el convertidor se utiliza:

```
make run
```

Si no se especifican parámetros, el sistema utiliza los valores predeterminados definidos en el Makefile:

- Expresión regular: (a|b)*abb
- Cadena de entrada: abb

También es posible proporcionar una expresión regular y una cadena diferentes mediante las variables ER y CADENA.

Ejemplo:

```
make run ER='(a|b)*abb' CADENA='abb'
```

Al ejecutarse, el programa presenta tres resultados principales:

- La tabla de transiciones del AFND.
- La tabla de transiciones del AFD.
- El recorrido de la cadena y el resultado de la evaluación.

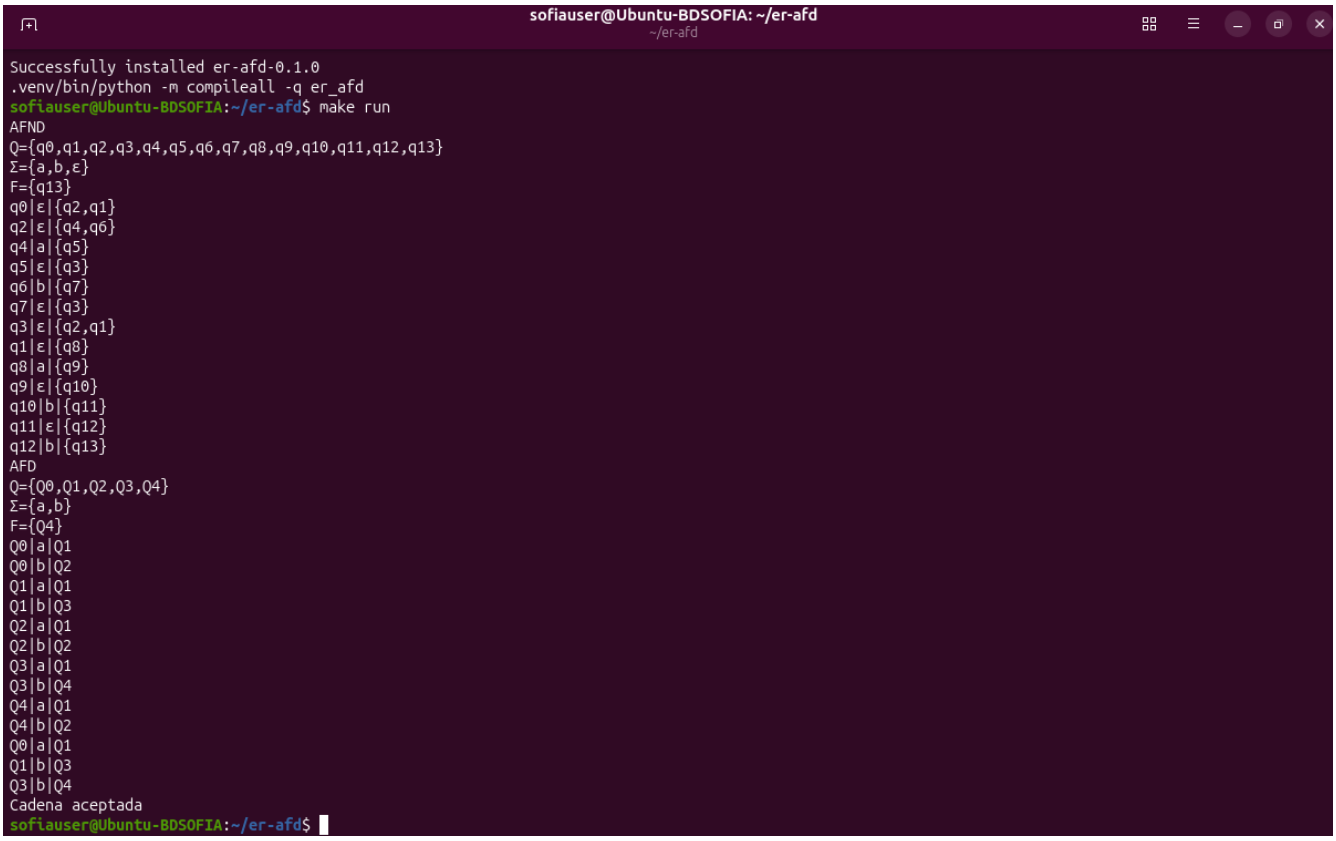


Figura 6. Captura real de la terminal de Ubuntu con test del Makefile

5.7. Ingreso de una expresión regular y otra cadena

El sistema permite modificar los valores de entrada sin necesidad de alterar el código fuente.

Para ello, se debe utilizar la siguiente estructura:

```
make run ER='EXPRESION_REGULAR' CADENA='CADENA'
```

Por ejemplo, para evaluar la expresión:

`(a|b)*abb`

utilizando la cadena:

`aabb`

se ejecuta:

```
make run ER='(a|b)*abb' CADENA='aabb'
```

El sistema construirá nuevamente los autómatas correspondientes a la expresión proporcionada y evaluará la cadena ingresada.

Es importante recordar que cada ejecución utiliza los parámetros proporcionados en ese momento.

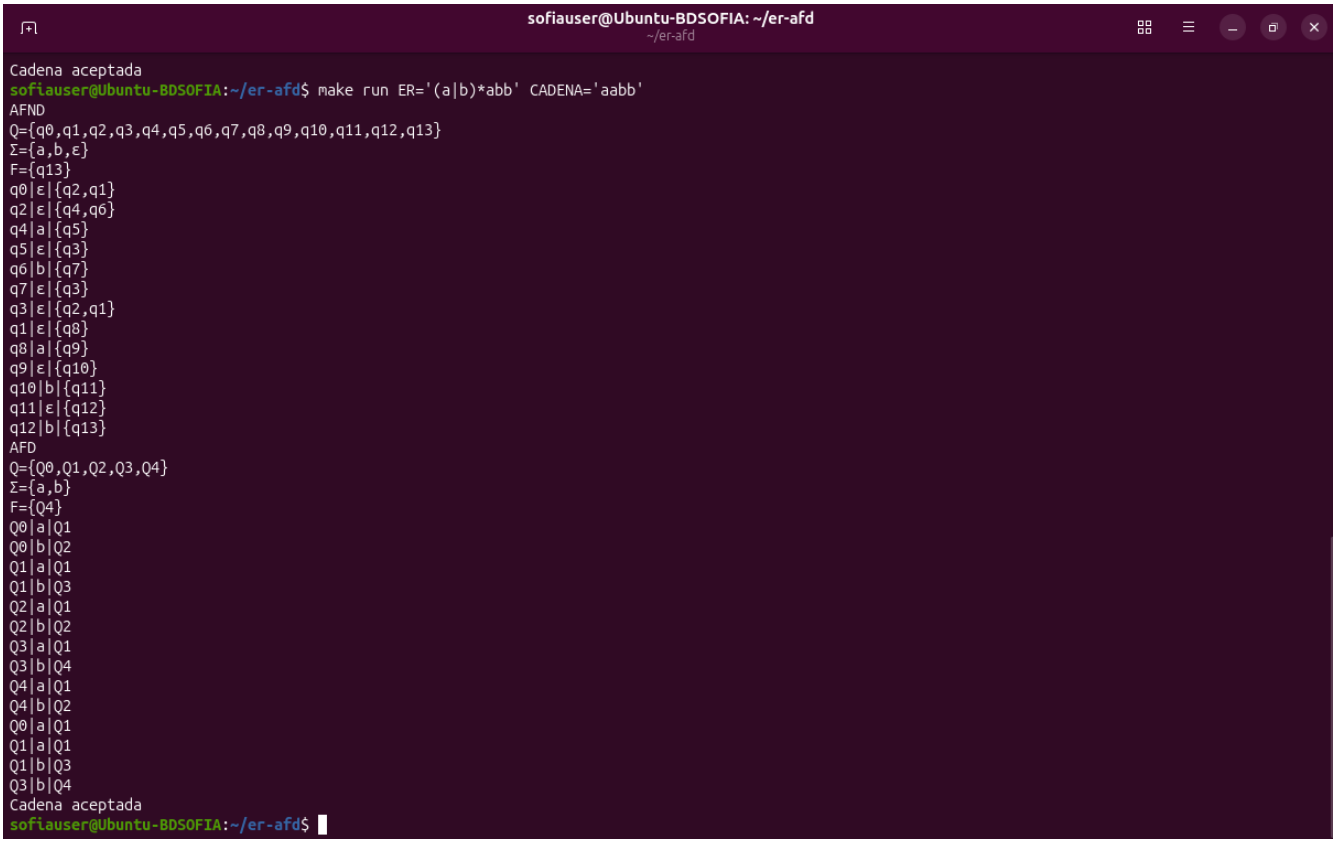


*Figura 7. Captura real de la terminal de Ubuntu ejemplo: ER='(a|b)*abb' CADENA='aabb'*

Si se ejecuta:

```
make run ER='hola\ mundo' CADENA='hola mundo'
```

Este ejemplo demuestra que el programa puede recibir entradas con espacios sin confundirlas con argumentos independientes.



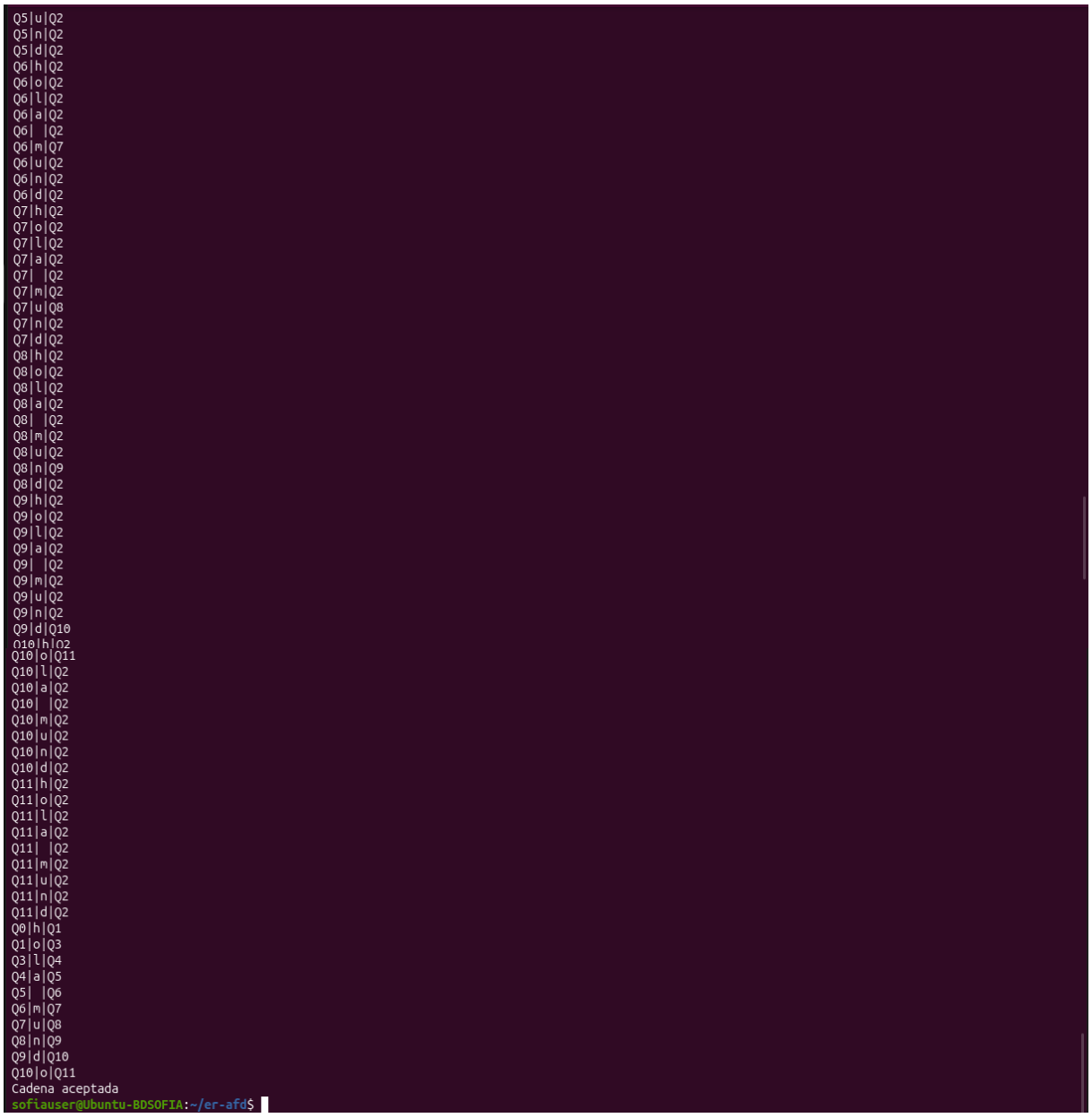


Figura 7. Captura real de la terminal de Ubuntu ejemplo: ER='hola\ mundo' CADENA='hola mundo'

5.8. Ejecución de las pruebas – make test

El comando:

```
make test
```

permite ejecutar las pruebas automatizadas almacenadas en el directorio tests/.

Estas pruebas verifican el funcionamiento de los diferentes componentes del sistema, incluyendo:

- Análisis y validación de expresiones regulares.

- Construcción del AFND.
- Conversión del AFND a AFD.
- Cálculo de cierres épsilon.
- Evaluación de cadenas.
- Reconocimiento de cadenas aceptadas y rechazadas.
- Manejo de espacios y caracteres especiales contemplados por el proyecto.

Al finalizar, Python presenta un resumen indicando la cantidad de pruebas ejecutadas y si estas finalizaron correctamente.

Cuando todas las pruebas son satisfactorias, se muestra el mensaje OK.

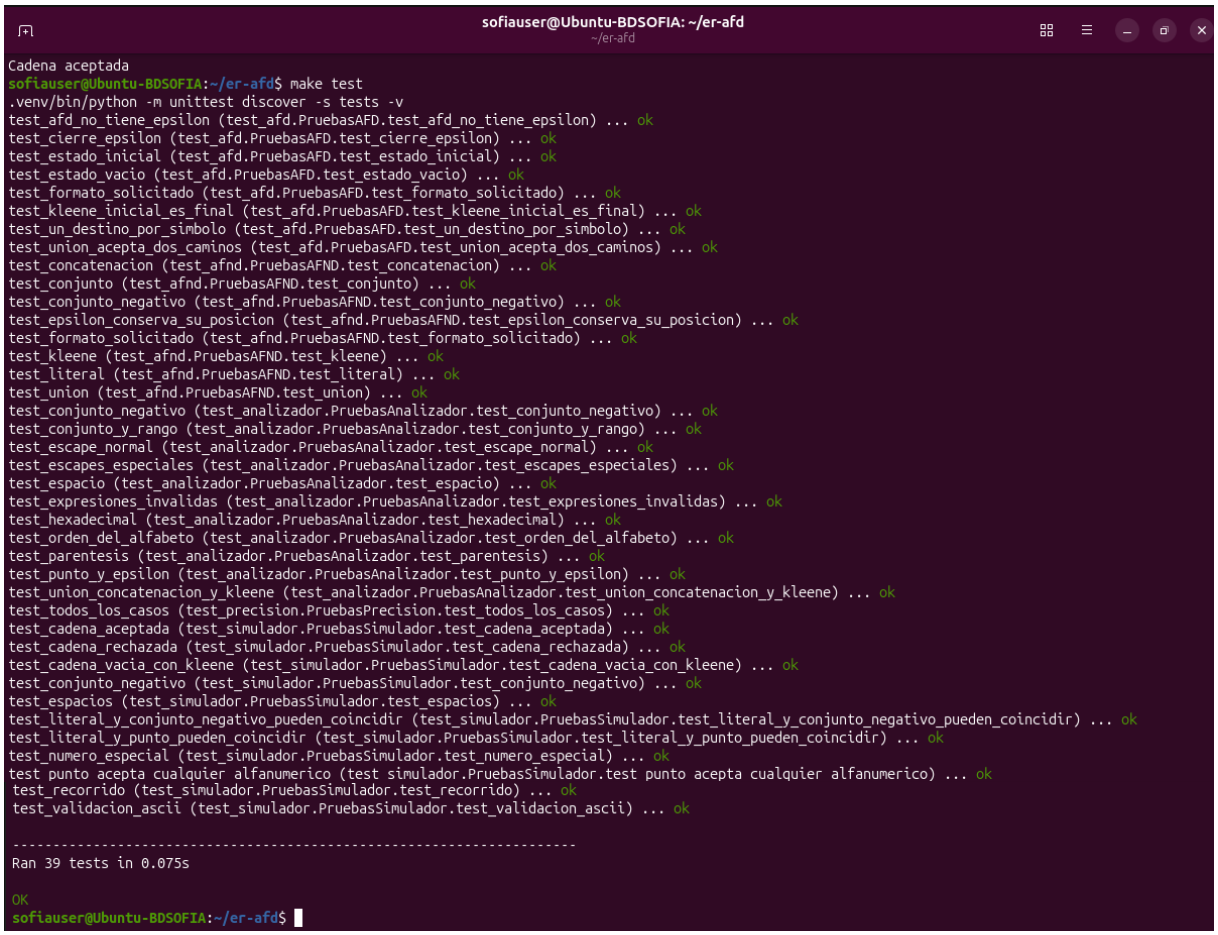


Figura 8. Captura real de la terminal de Ubuntu con las pruebas automatizadas

5.9. Limpieza del proyecto – make clean

El comando:

```
make clean
```

permite eliminar los archivos y directorios temporales generados durante la configuración y ejecución del proyecto.

Entre los elementos eliminados se encuentran:

- .venv: entorno virtual de Python.
- .pytest_cache: archivos temporales de pruebas.
- build/ y dist/: directorios de construcción y distribución.
- Archivos *.egg-info: metadatos generados durante la instalación.
- Directorios __pycache__: archivos de caché de Python.

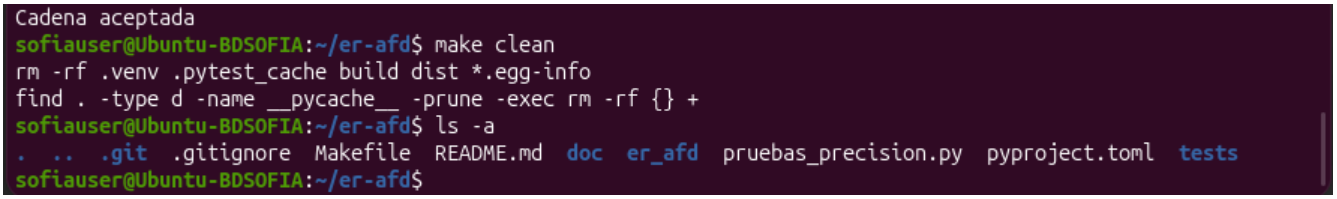


Figura 9. Captura real de la terminal de Ubuntu con .venv eliminado

Este procedimiento permite dejar el proyecto limpio, eliminando componentes generados localmente, sin modificar los archivos fuente ni las pruebas originales.

Para volver a preparar el sistema después de la limpieza, se debe ejecutar:

`make build`

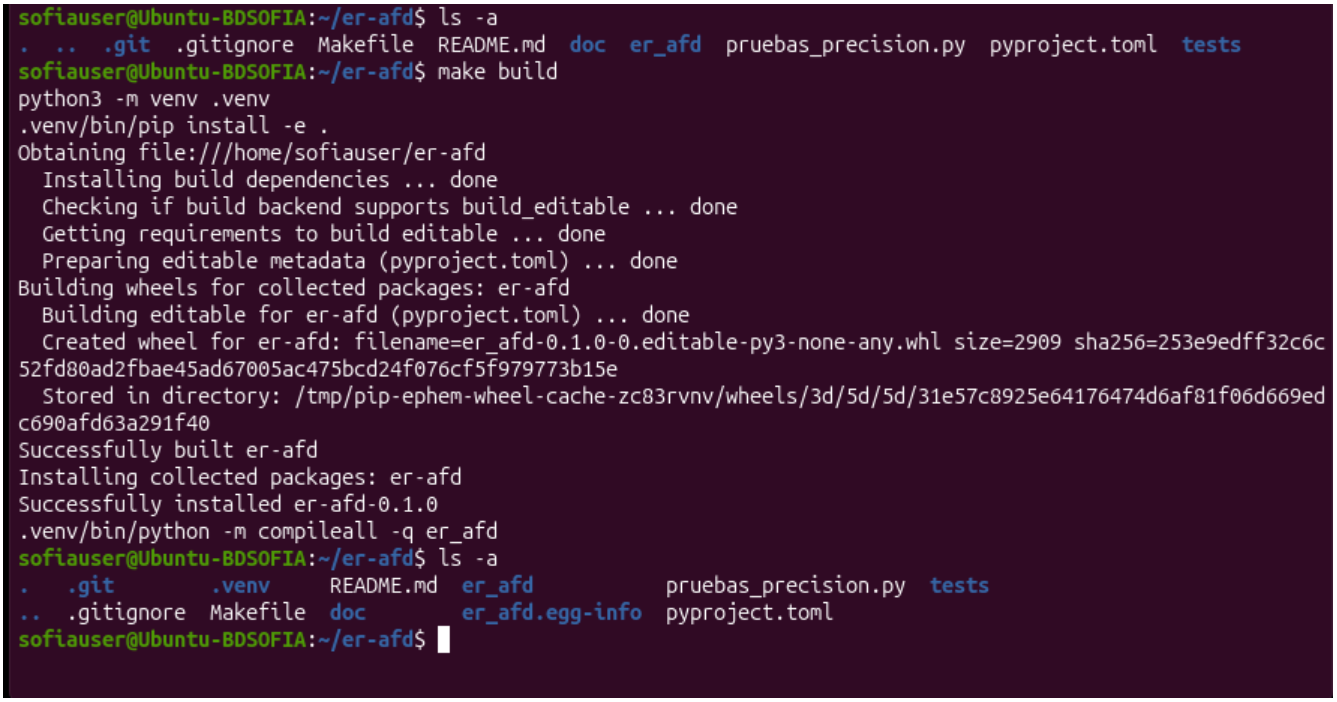


Figura 10. Captura real de la terminal de Ubuntu con los archivos restaurados `make build`

5.10. Interpretación de los resultados

Una vez ejecutado el programa, se presentan las siguientes secciones:

- **A) AFND – Autómata Finito No Determinista**

Representa la primera conversión de la expresión regular mediante el algoritmo de Thompson.

Se muestran:

- Q: conjunto de estados del autómata.
- Σ: alfabeto de entrada, incluyendo las transiciones épsilon.
- F: conjunto de estados finales.
- q0: estado inicial.
- Tabla de transiciones: indica el estado de origen, el símbolo leído y los estados de destino.

Los estados se identifican mediante la letra q seguida de un número correlativo.

Las transiciones épsilon (ε) permiten cambiar de estado sin consumir un carácter de la cadena de entrada.

- **B) AFD – Autómata Finito Determinista**

Representa el resultado de convertir el AFND mediante la construcción de subconjuntos.

Se muestran:

- Q: conjunto de estados deterministas.
- Σ: alfabeto de entrada.
- F: estados de aceptación.
- Q0: estado inicial.
- Tabla de transiciones: establece un único estado de destino para cada combinación de estado y símbolo.

Los estados del AFD se identifican mediante la letra Q seguida de un número correlativo.

Cada estado del AFD representa un conjunto de estados del AFND. Los estados finales son aquellos cuyo conjunto contiene el estado final del AFND.

- **C) Recorrido de la cadena**

Después de construir el AFD, el sistema evalúa la cadena ingresada carácter por carácter, iniciando desde `Q0.`

El recorrido muestra las transiciones realizadas, utilizando el siguiente formato:

```
Q0|a|Q1
Q1|b|Q2
```

Cada línea representa el estado actual, el carácter procesado y el estado al que se transiciona.

- **D) Resultado final**

Cuando se termina de procesar la cadena, el sistema comprueba si el último estado alcanzado pertenece al conjunto de estados finales.

Los resultados posibles son:

- Cadena aceptada: la cadena termina en un estado de aceptación del AFD.
- Cadena rechazada: la cadena no termina en un estado de aceptación.

El programa utiliza el código de retorno 1 para una cadena aceptada y 0 para una cadena rechazada, de acuerdo con las especificaciones del proyecto.

5.11. Uso de comillas y espacios en las entradas

Cuando una expresión regular o una cadena contiene espacios, es necesario utilizar comillas dobles para agrupar la entrada o representar el espacio mediante una secuencia de escape.

Opción 1. Utilizando comillas simples para los argumentos del comando:

```
make run ER='hola\ mundo' CADENA='hola mundo'
```

En este ejemplo, la expresión regular contiene un espacio representado mediante `\ `, mientras que la cadena de entrada contiene un espacio literal protegido por las comillas.

Opción 2. Utilizando comillas dobles para agrupar una entrada con espacios:

En la ejecución directa del programa se pueden utilizar comillas dobles:

```
.venv/bin/er-afd 'hola\ mundo' "hola mundo"
```

En ambos casos, el sistema interpreta la expresión regular y la cadena como argumentos completos, evitando que los espacios sean considerados separadores entre parámetros.

El resultado esperado para este ejemplo es:

```
Cadena aceptada
```

5.12. Finalización

Con estos procedimientos, el usuario puede instalar, configurar, ejecutar y comprobar el funcionamiento del convertidor ER-AFD desde una terminal Linux, así como realizar nuevas evaluaciones utilizando diferentes expresiones regulares y cadenas de entrada.

El sistema permite visualizar las etapas de conversión y el proceso de reconocimiento, facilitando la comprensión práctica del funcionamiento de los autómatas finitos.

10. Conclusiones

- El desarrollo del proyecto ER-AFD permitió comprender y aplicar de manera práctica los fundamentos de la teoría de autómatas y lenguajes formales, demostrando cómo una expresión regular puede transformarse en un modelo computacional capaz de reconocer cadenas de caracteres. Mediante esta implementación se relacionaron los conceptos teóricos de expresiones regulares, autómatas finitos no deterministas y autómatas finitos deterministas.
- La implementación del algoritmo de Thompson permitió construir un Autómata Finito No Determinista (AFND) a partir de una expresión regular, utilizando estados y transiciones épsilon para representar operaciones como la unión, concatenación y cerradura de Kleene. Esta etapa facilita visualizar cómo se estructura el autómata que representa el lenguaje definido por la expresión regular.
- La conversión del AFND a un Autómata Finito Determinista (AFD), mediante el método de construcción de subconjuntos, permitió obtener una representación determinista del mismo lenguaje. Este procedimiento demuestra la equivalencia entre ambos modelos de autómatas y facilita la evaluación de cadenas mediante transiciones definidas para cada símbolo del alfabeto.
- La implementación del módulo de simulación permitió comprobar el funcionamiento de los autómatas mediante el procesamiento de cadenas de entrada carácter por carácter. La visualización del recorrido de estados y del resultado final facilita identificar si una cadena pertenece o no al lenguaje reconocido por el autómata.
- La utilización de herramientas como Python, Git y GNU Make permitió organizar y automatizar las actividades de configuración, construcción, ejecución, pruebas y limpieza del proyecto. Asimismo, el uso de un entorno virtual contribuyó a mantener aisladas las dependencias necesarias para la instalación y ejecución del sistema.
- La ejecución de las pruebas automatizadas permitió verificar el comportamiento de los diferentes módulos del programa, mientras que los comandos `make clean` y `make build` demostraron que es posible eliminar los archivos generados durante la ejecución y reconstruir posteriormente el entorno de trabajo sin modificar los archivos fuente del proyecto.
- Finalmente, la elaboración de esta guía de instalación y uso permitió documentar los procedimientos necesarios para que otros usuarios puedan obtener el código fuente, configurar el entorno, ejecutar el convertidor y analizar sus resultados. De esta manera, ER-AFD constituye una herramienta de apoyo para el aprendizaje práctico de los procesos de conversión de expresiones regulares a autómatas finitos.

11. Referencias

- Universidad Mariano Gálvez de Guatemala. (2026). ER-AFD: Convertidor de ER a AFD [Enunciado del proyecto, septiembre de 2026].
- DanielM73. (s. f.). *ER-AFD: Convertidor de expresiones regulares a autómatas finitos deterministas* [Repositorio de código fuente]. GitHub. https://github.com/DanielM73/er-afd
- Equipo de desarrollo ER-AFD. (2026). Documentación interna: doc/01_modulo_entrada_er.md, doc/02_construccion_afnd.md, doc/03_conversion_afd.md y doc/04_simulacion_cadena.md.
- Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, techniques, and tools* (2nd ed.). Pearson.
- Python Software Foundation. (s. f.). *Python 3 documentation*. https://docs.python.org/3/
- Thompson, K. (1968). Programming techniques: Regular expression search algorithm. *Communications of the ACM, 11*(6), 419–422. https://doi.org/10.1145/363347.363387
- University of Illinois Urbana-Champaign. (2021). *Models of computation: Lecture 4 – Nondeterministic automata*. https://courses.physics.illinois.edu/cs374/fa2021/A/notes/models/04-nfa.pdf
- GNU Project. (s. f.). *GNU Make manual*. https://www.gnu.org/software/make/manual/